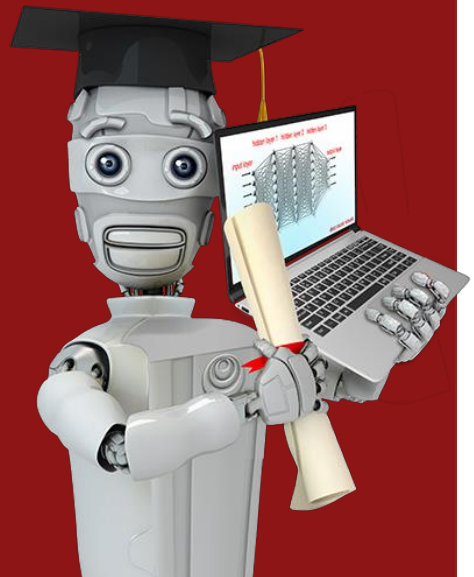
Copyright Notice

🧠 The Ultimate Master's Summary Table

Architecture	Inductive Bias	Risk	Best Used For
Constant Width	Preserve feature dimensionality while refining abstraction	High compute, but stable	Large Language Models, Residual Networks
Decreasing Width	Forced compression / Information bottleneck	Losing critical information (underfitting)	Autoencoders, Dimensionality Reduction
Increasing Width	Expansion to high-dimensional space	Massive overfitting, high compute cost	Decoders, Generative models
Deep + Narrow (Same total params)	Sequential, compositional step-by-step reasoning	Vanishing gradients, slow convergence	Structured data, vision (CNNs), speech
Shallow + Wide (Same total params)	Direct, brute-force universal mapping	Memorization (overfitting), no reuse of features	Small tabular datasets, simple XOR problems

DeepLearning.AI

Stanford
ONLINE



Advanced Learning Algorithms

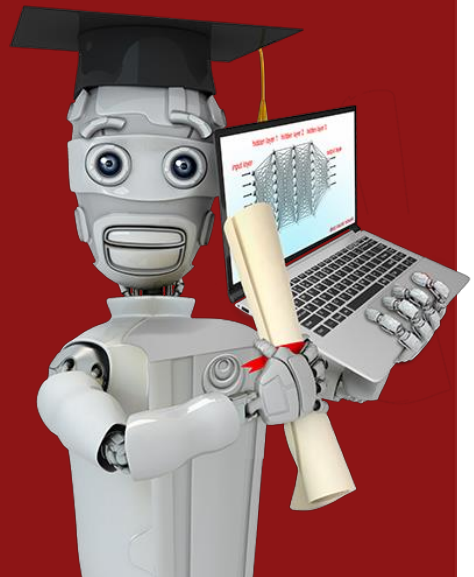
Welcome!

Advanced learning algorithms

Neural Networks 
  inference (prediction)
  training
Practical advice for building machine learning systems 
Decision Trees 

DeepLearning.AI

Stanford
ONLINE



Neural Networks Intuition

Neurons and the brain

Neural networks

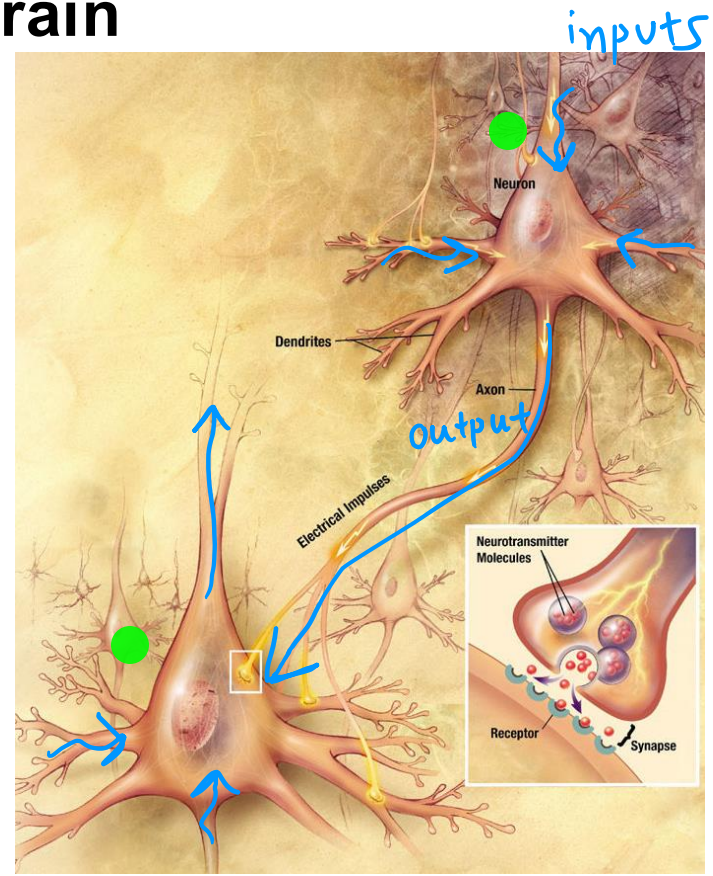
Origins: Algorithms that try to mimic the brain.

Used in the 1980's and early 1990's.
Fell out of favor in the late 1990's.

Resurgence from around 2005.

speech → images → text (NLP) → ...

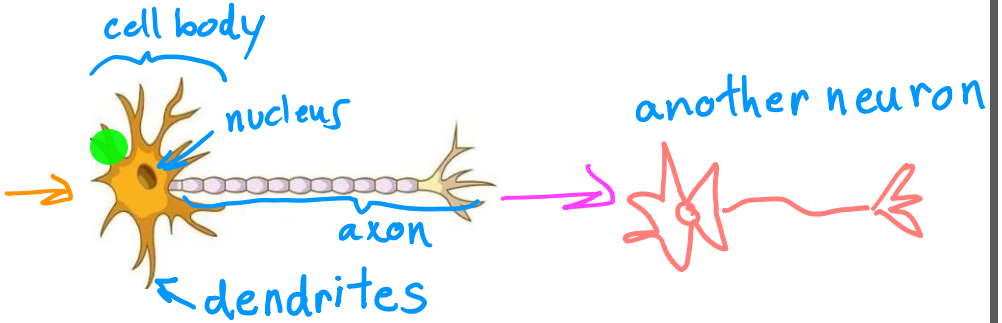
Neurons in the brain



[Credit: US National Institutes of Health, National Institute on Aging]

Biological neuron

inputs outputs



Simplified mathematical model of a neuron

inputs outputs

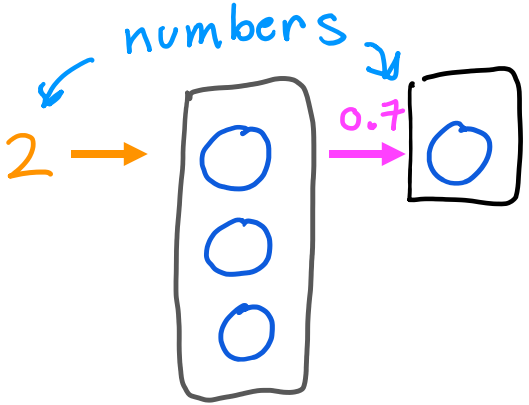
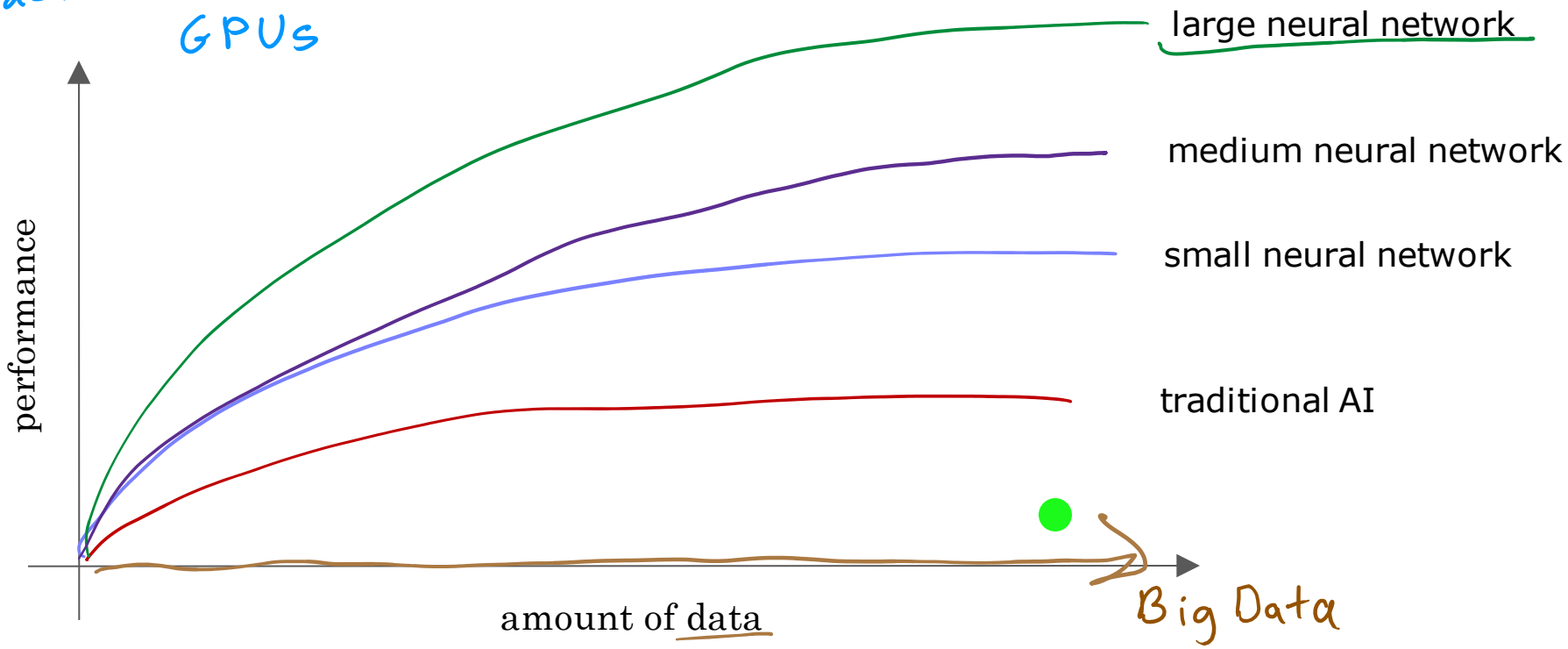


image source: https://biologydictionary.net/sensory-neuron/

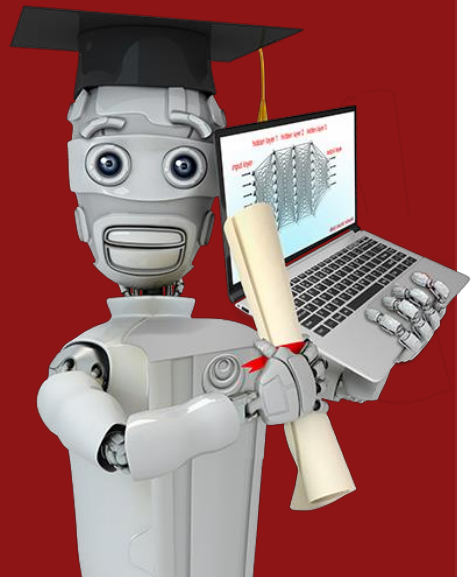
Why Now?

Faster computer processors
GPUs



DeepLearning.AI

Stanford ONLINE

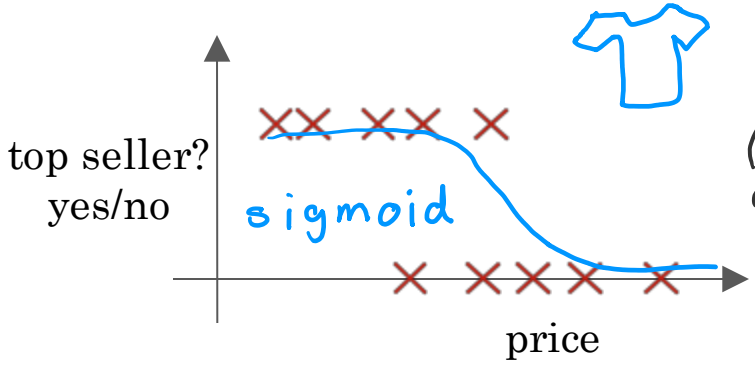


Neural Network Intuition

Demand Prediction

Hidden Layers = Feature Extractors: Each layer transforms the input vector into a new, more useful representation.

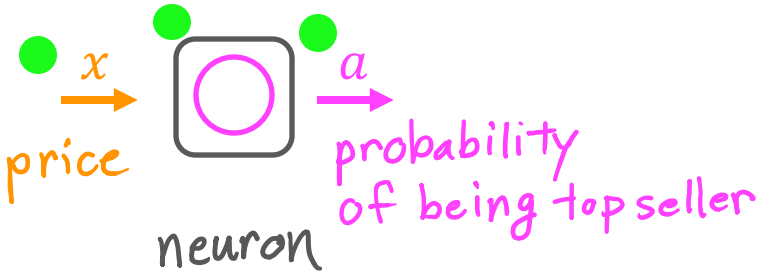
Demand Prediction



$x = price$ input

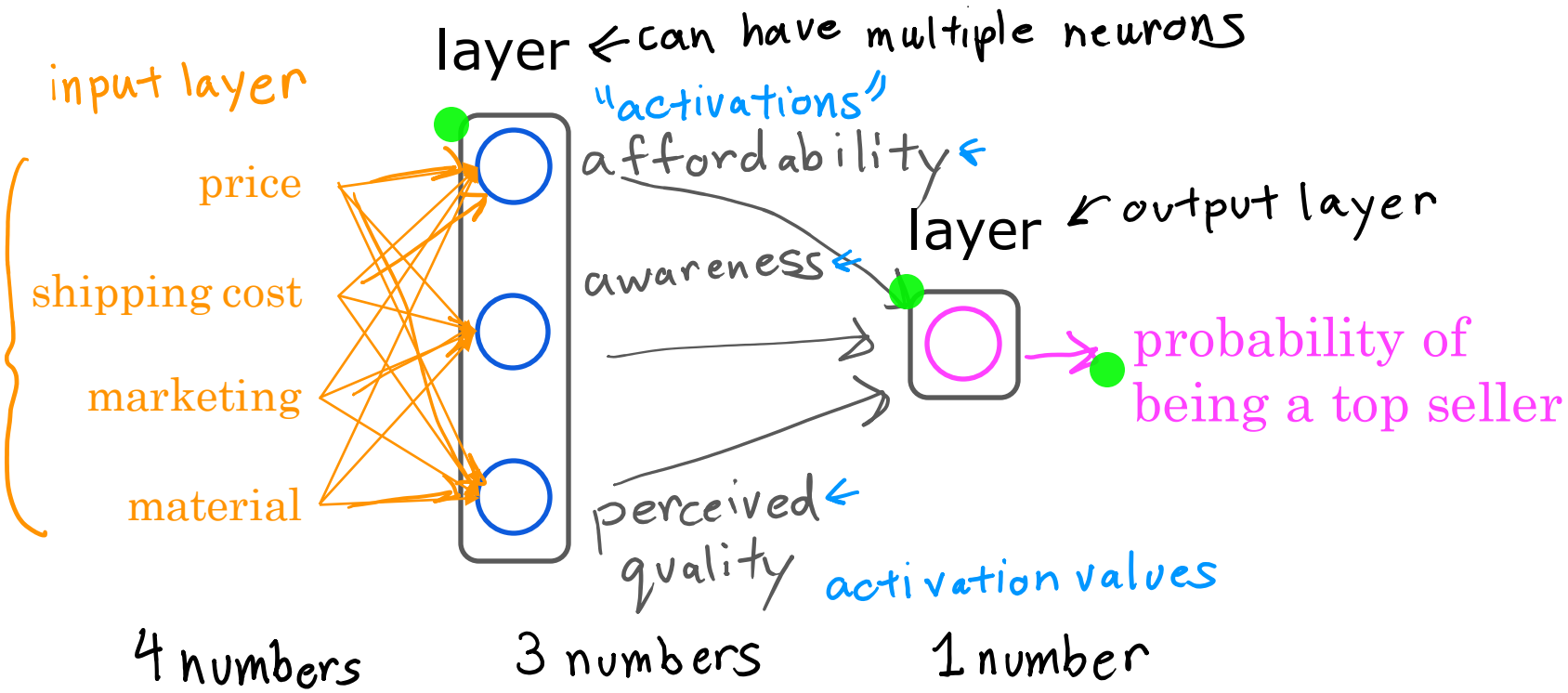
$a = f(x) = \frac{1}{1 + e^{-(wx+b)}}$ output

activation

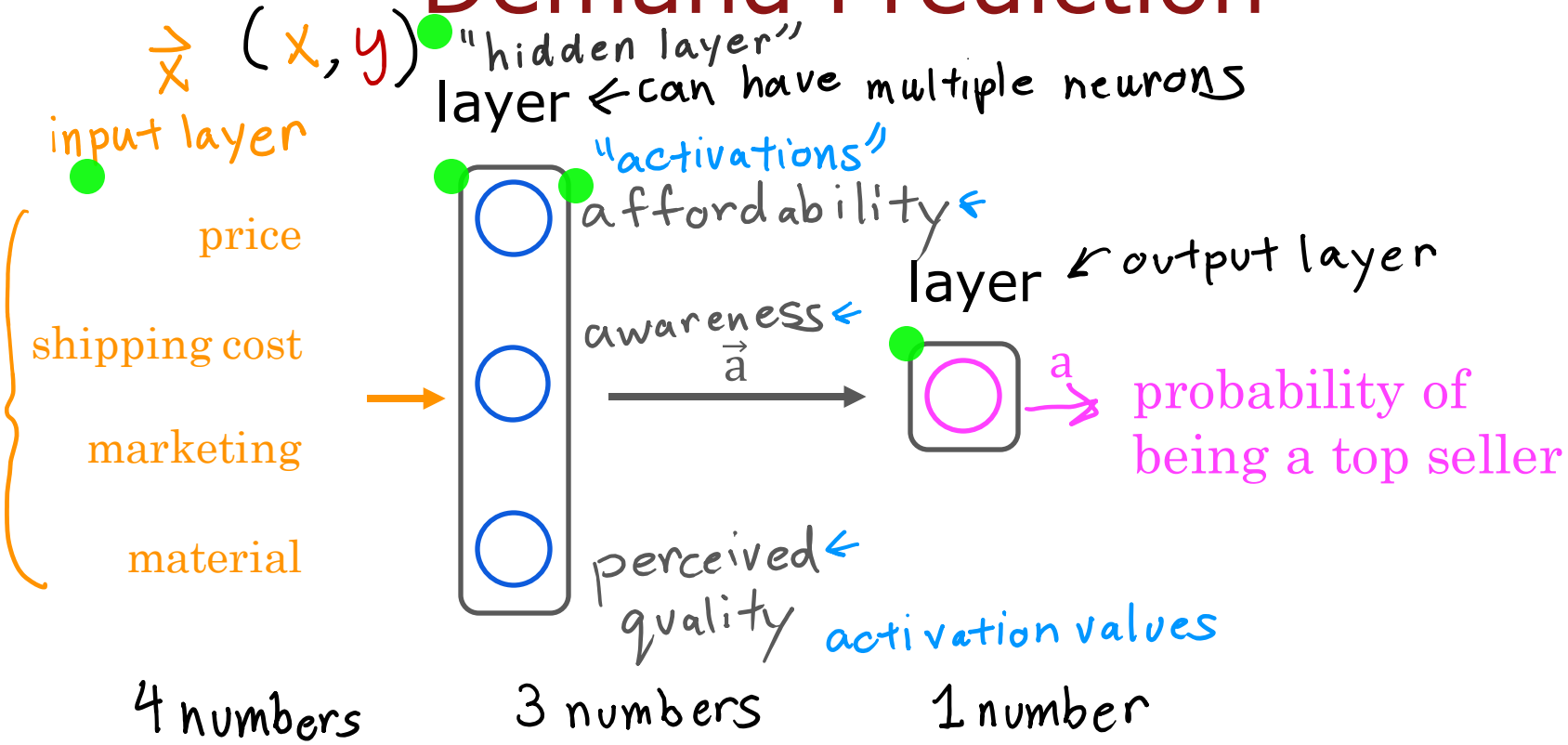


The network learns a hierarchy of representations. Early layers learn very simple patterns (e.g., edges in images, or basic linear combinations of price/shipping), while deeper layers learn highly abstract, task-specific concepts (e.g., "affordability" or "face shape") that maximize the predictive power for the target y. This is known as Representation Learning.

Demand Prediction



Demand Prediction



Demand Prediction

feature engineering

x_1 x_2

$\vec{x}$ (x, y)

input layer

price

shipping cost

marketing

material

4 numbers

"hidden layer"

layer ← can have multiple neurons

"activations"

affordability ←

awareness ←

perceived quality ←

$\vec{a}$

layer ← output layer

a

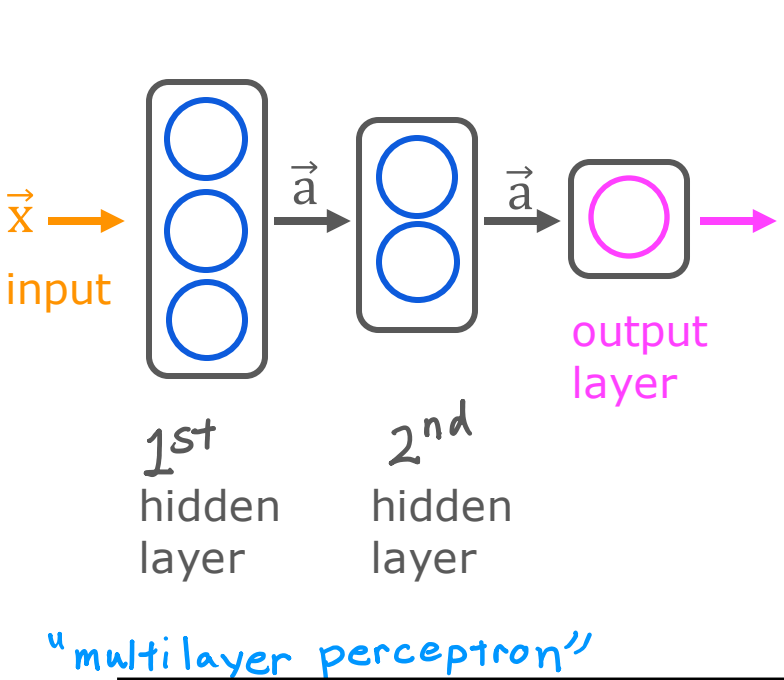
probability of being a top seller

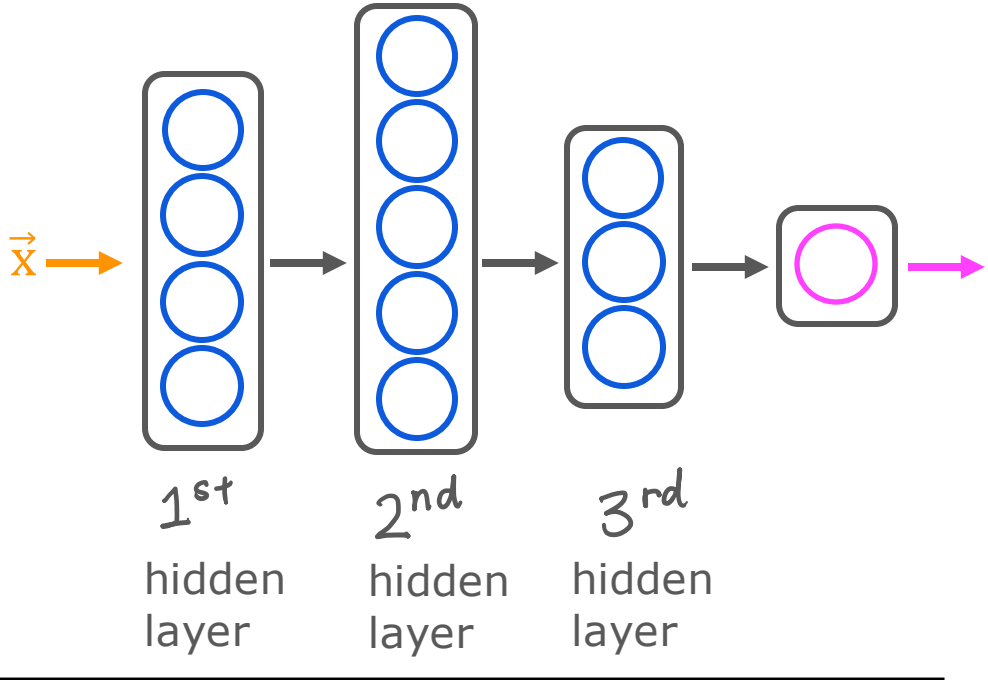
activation values

3 numbers

1 number

Multiple hidden layers



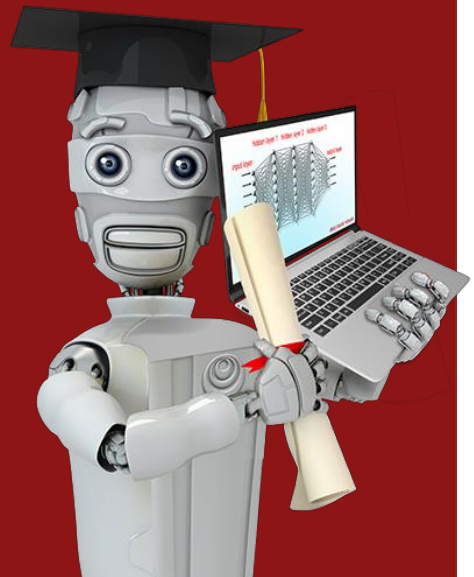


Think of it as having multiple processing stages. (e.g., In image recognition: Pixels Edges Textures Parts of objects Whole object).

neural network architecture

DeepLearning.AI

Stanford
ONLINE



The network learns a hierarchy of representations.
Early layers learn very simple patterns (e.g., edges in images, or basic linear combinations of price/shipping), while deeper layers learn highly abstract, task-specific concepts
(e.g., "affordability" or "face shape") that maximize the predictive power for the target y.
This is known as Representation Learning.

Neural Networks Intuition

Example: Recognizing Images

Face recognition

1000 pixels

1000 pixels

1000 columns

1000 rows

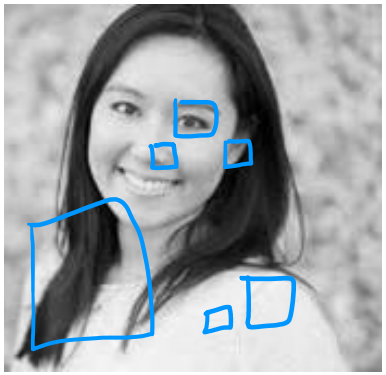
197	185	203	...	...
...	57	64	92	...
⋮				
		...	187	214

$\vec{x} =$

197
185
203
⋮
57
64
92
⋮
187
214

1 million

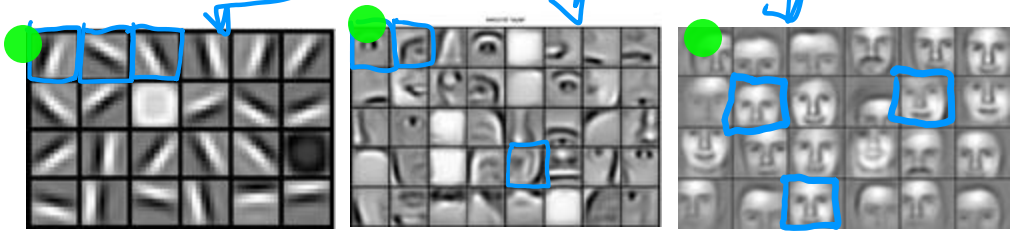
Face recognition



$\vec{x}$
input

output layer

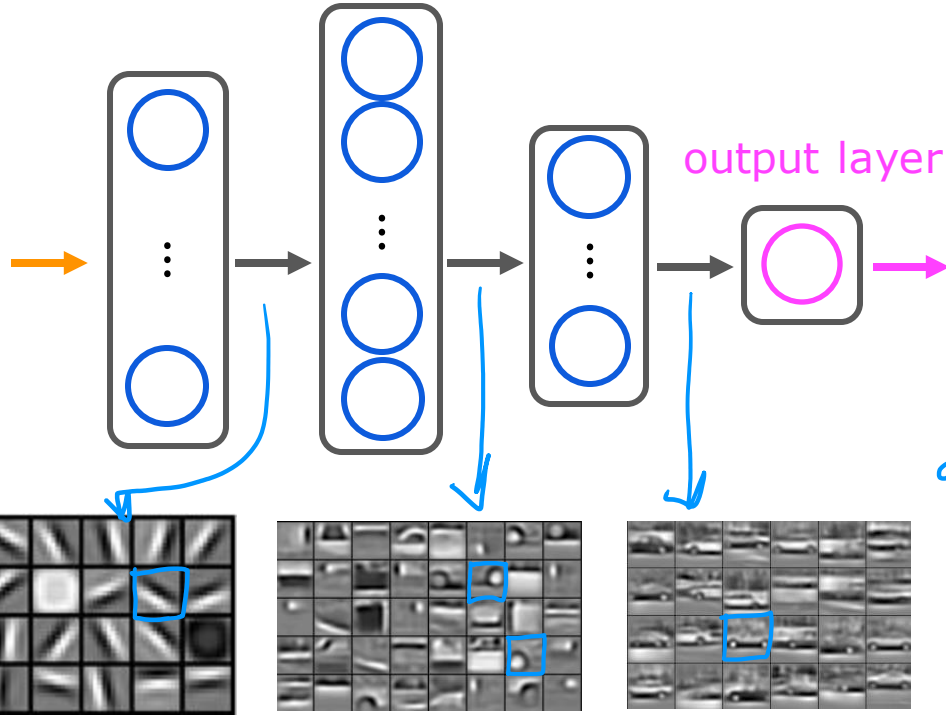
probability of being person 'XYZ'



activations are higher level features

source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

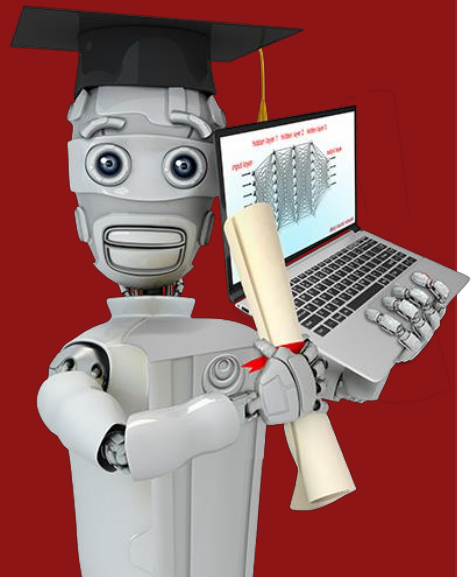
Car classification



source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

DeepLearning.AI

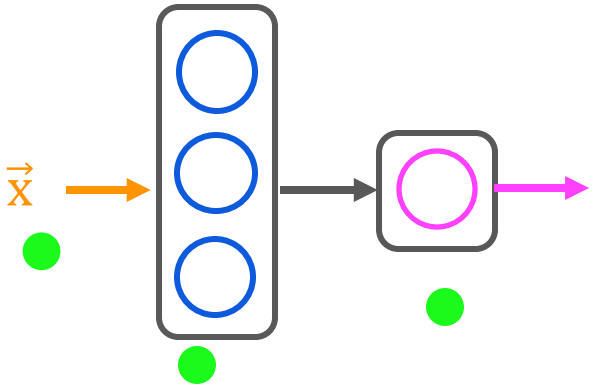
Stanford
ONLINE



Neural network model

Neural network layer

Neural network layer



1. The Superscript Notation (The "Layer ID")

Andrew introduces the **square bracket** `[l]` notation. This is the universal language of deep learning papers.

- **The Rule**: `a^{[l]}` = the activation vector from **layer l**.
- **Your Brain Hack**: Think of the superscript `[l]` as the **floor number** in a building. Floor 0 (input) sends data up to Floor 1, which sends it up to Floor 2.

Symbol	Meaning	In the Transcript's Example
`x` or `a^{[0]}`	Input features (Layer 0)	Price, Shipping, Marketing, Material (4 numbers)
`a^{[1]}`	Output of Layer 1 (Hidden)	Vector `[0.3, 0.7, 0.2]` (3 numbers)
`a^{[2]}`	Output of Layer 2 (Output)	Scalar `0.84` (1 number)
`w^{[1]}_2`	Weight for the **2nd neuron** in **Layer 1**	Connects 4 inputs to that specific neuron
`b^{[1]}_3`	Bias for the **3rd neuron** in **Layer 1**	The offset for that logistic unit

> **Master's Insight**: Notice that `w^{[1]}_1` is a **vector** (because it has to multiply the *4* input features), while `a^{[1]}` is a **vector** (containing the 3 outputs). When you move to Layer 2, `w^{[2]}_1` is a **vector of size 3** (because it multiplies the 3 inputs from Layer 1) to produce `a^{[2]}` (scalar).

3. The "Fully Connected" Reality (Revisiting your "Reduce Irrelevant" comment)

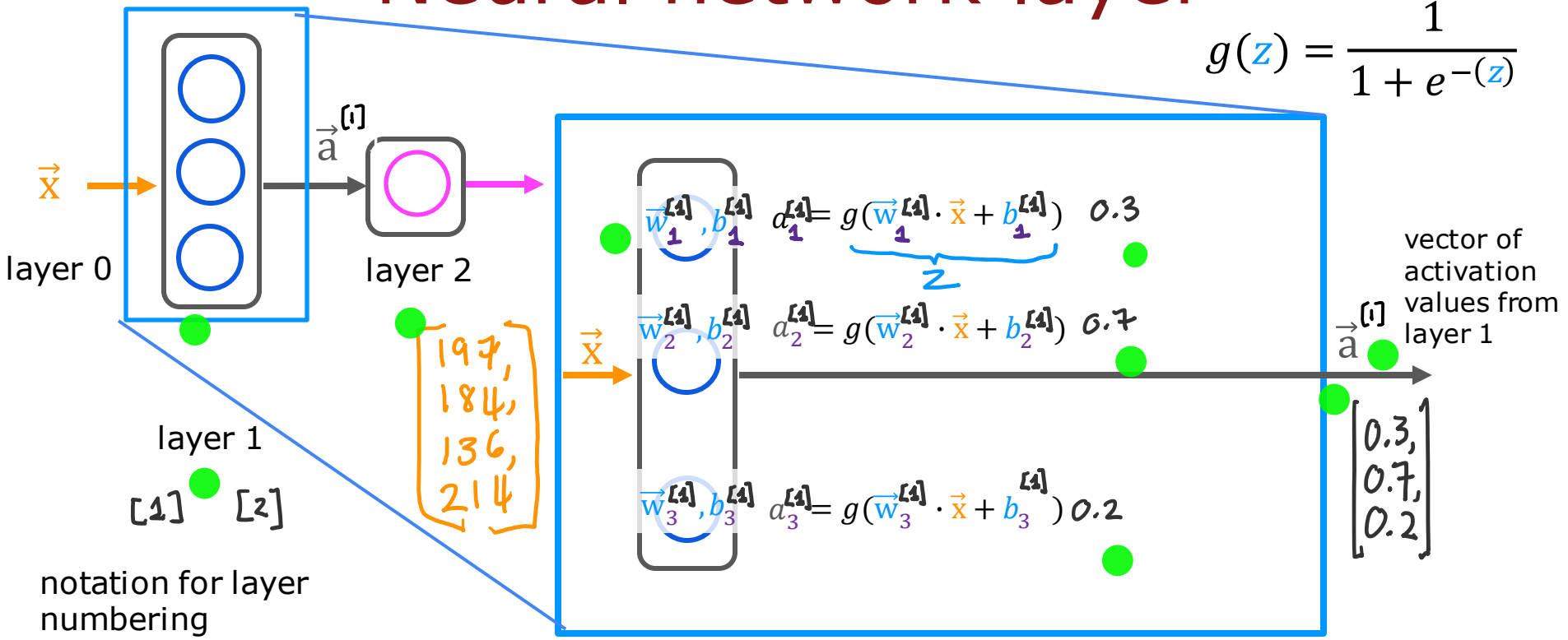
Notice that Andrew explicitly says **every input connects to every neuron**.
Why? Because even if a neuron is *trying* to predict "affordability" (price + shipping), we still feed it "marketing" and "material".

- **The Magic**: The network learns to set the weights for "marketing" and "material" in that specific neuron to **~0** during training.
- **Your previous thought ("reduce irrelevant one by one") was actually spot-on!** The network doesn't delete the feature; it **annihilates the weight** via gradient descent, effectively killing that connection for that specific neuron, while other neurons keep their weights high for those features.

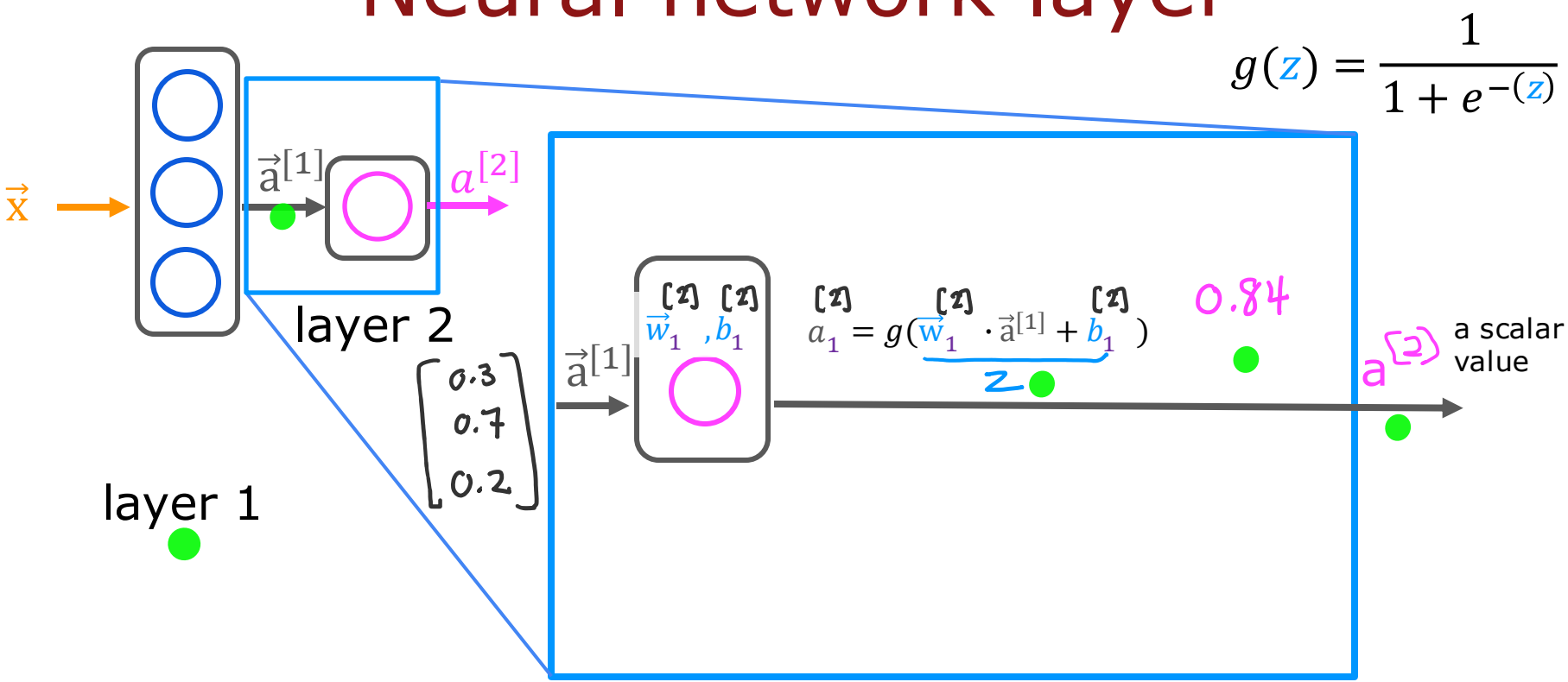
Neural network layer

Vanishing Gradient Problem ???

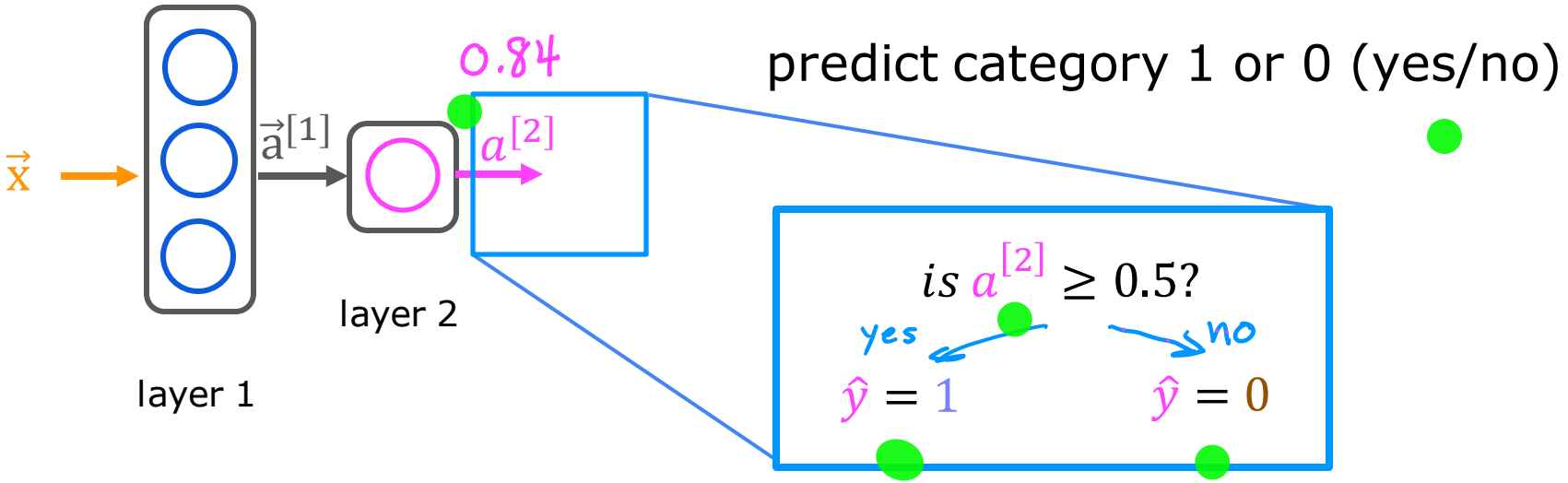
$$g(z) = \frac{1}{1 + e^{-(z)}}$$



Neural network layer



Neural network layer

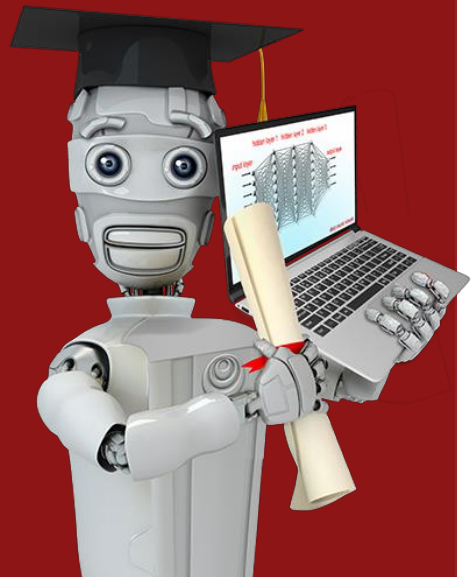


Here is the exact flow from the transcript, expressed in the cleanest Master's formula:

1. **Input:** $A^{[0]} = X$ (Shape: 4×1)
2. **Layer 1 (Hidden):** $Z^{[1]} = W^{[1]}A^{[0]} + b^{[1]} \rightarrow A^{[1]} = \sigma(Z^{[1]})$ (Shape: 3×1)
3. **Layer 2 (Output):** $Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]} \rightarrow A^{[2]} = \sigma(Z^{[2]})$ (Shape: 1×1)
4. **Prediction:** $\hat{y} = 1$ if $A^{[2]} \geq 0.5$ else 0.

DeepLearning.AI

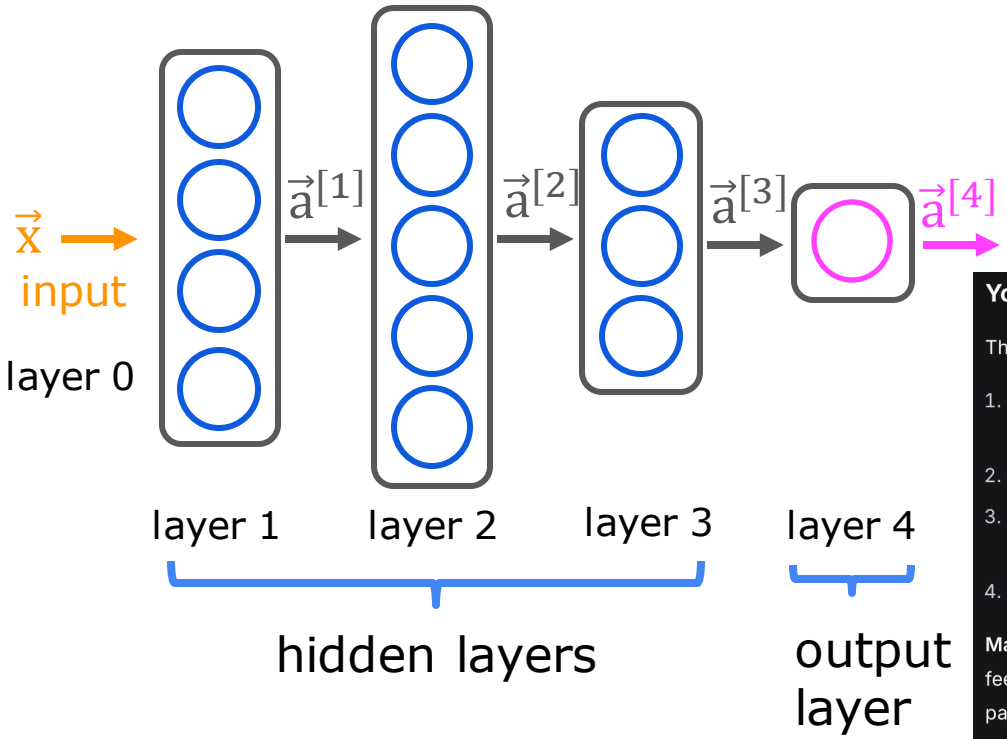
Stanford
ONLINE



Neural Network Model

More complex neural networks

More complex neural network



Your Ultimate "Sequential Stage" Cheat Sheet

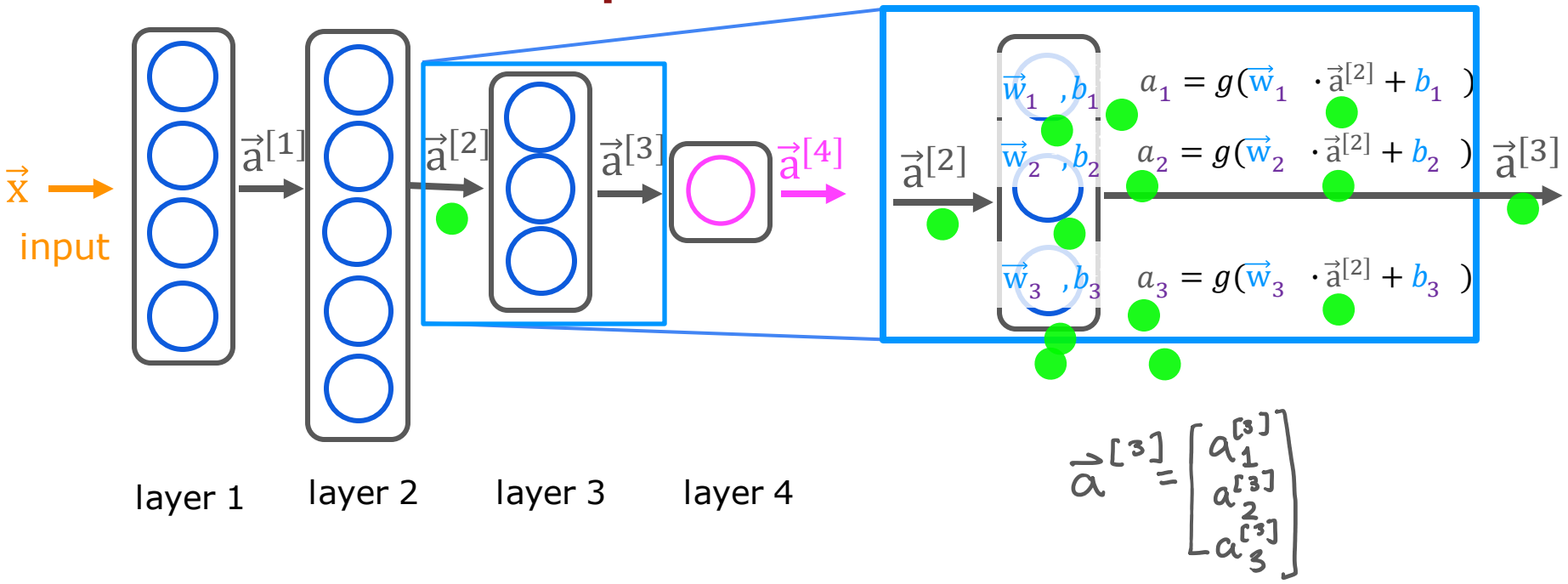
Think of this 4-layer network as a 4-step assembly line:

1. **Stage 1 (Layer 1)**: Takes raw features (price, shipping, marketing, material). Outputs 5 basic combined features.
2. **Stage 2 (Layer 2)**: Takes those 5 basic features. Outputs 5 slightly more abstract features.
3. **Stage 3 (Layer 3)**: Takes those abstract features. Compresses them down to 3 highly sophisticated concepts (maybe actual "affordability", "awareness", "quality").
4. **Stage 4 (Layer 4 / Output)**: Takes those 3 concepts. Spits out the final probability (0.84).

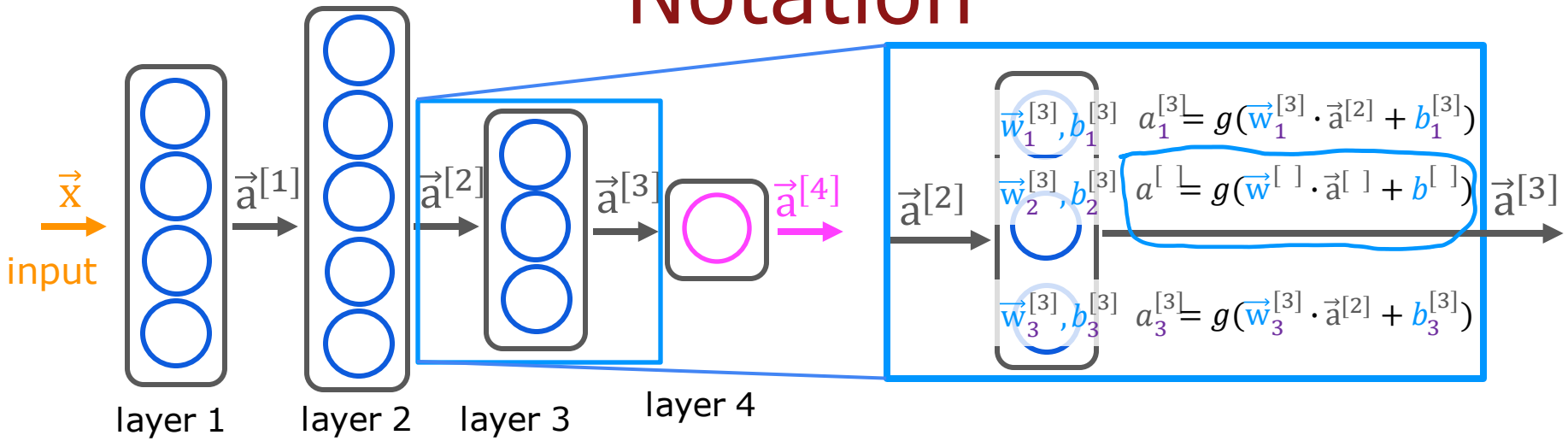
Master's Takeaway: You now have the exact mathematical vocabulary to describe *any* feedforward neural network. The notation $a_j^{[l]}$ is the universal language of every deep learning paper you will ever read.

Since we've now covered the math notation *perfectly*, shall we move on to the **Vectorized Implementation** (how to actually code this in NumPy without loops) or **Backpropagation** (how the network actually learns those W and b values)?

More complex neural network



Notation



6. The Vectorization Reality Check (For your code)

Andrew writes $w \cdot a$ for a single neuron. But in real code, we **never** loop over neurons. We stack the weights of *all* neurons in Layer l into a **Matrix** $W^{[l]}$.

- **Shape of $W^{[l]}$**: (Number of neurons in layer l × Number of neurons in layer $l-1$).
- **The actual vectorized computation** (which Andrew will teach next) is:

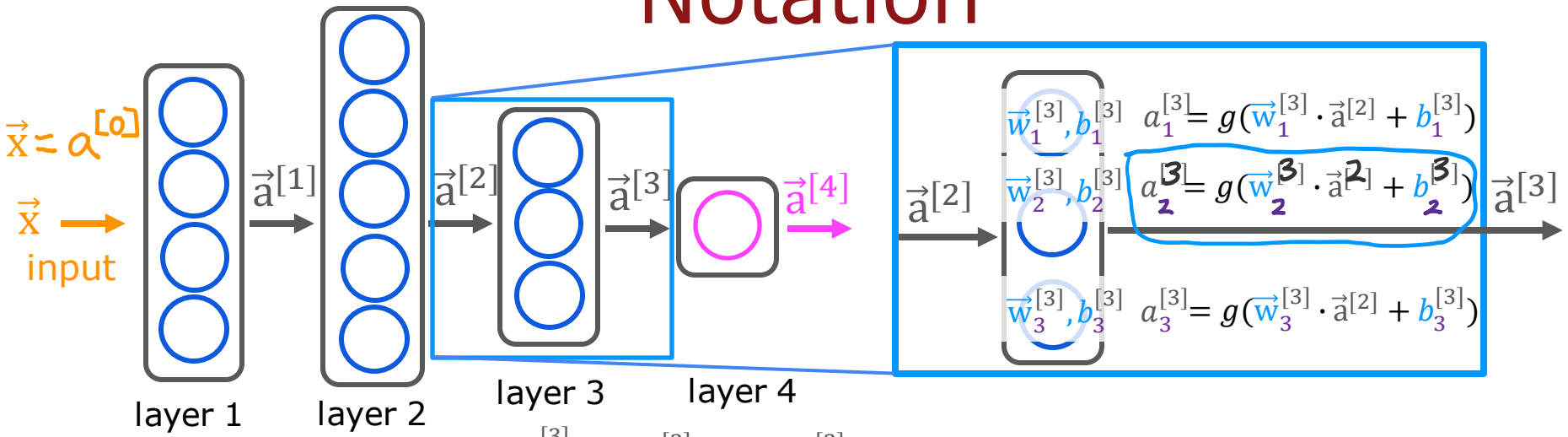
$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g(Z^{[l]})$$

Where $A^{[l]}$ becomes a vector containing $a_1^{[l]}, a_2^{[l]}, a_3^{[l]}$ all at once!

Question:
Can you fill in the superscripts and subscripts for the second neuron?

Notation



$$a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$$

Activation value of layer l, unit(neuron) j

output of layer $l-1$ (previous layer)

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})$$

sigmoid "activation function"

Parameters w & b of layer l, unit j

1. The Golden Shape Rule (Memorize this!)

For the multiplication $W \cdot A$ to work, the **inner dimensions** must match and **cancel out**. The two **outer dimensions** are the ones that survive in the result.

$$\text{Result } Z = \underbrace{W}_{(\text{Neurons} \times \text{Features})} \cdot \underbrace{A}_{(\text{Features} \times \text{Batch Size})} + \underbrace{b}_{(\text{Neurons} \times 1)}$$

Plug in your numbers (5 neurons, 64 pixels, 100 images):

$$\underbrace{Z}_{(5 \times 100)} = \underbrace{W}_{(5 \times 64)} \cdot \underbrace{A}_{(64 \times 100)} + \underbrace{b}_{(5 \times 1)}$$

Notice: The 64 (features) is in the middle and gets multiplied and summed away. The 5 (neurons) and 100 (batch) are on the outside, so they define the final shape.

Layer 1 (64 inputs → 5 neurons):

$Z^{[1]} = W^{[1]} \cdot A^{[0]} + b^{[1]}$

$(5 \times 100) = (5 \times 64) \cdot (64 \times 100) + (5 \times 1)$

Parameters: $(64 \times 5) + 5 = 325$

Layer 2 (5 inputs → 3 neurons):

$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$

$(3 \times 100) = (3 \times 5) \cdot (5 \times 100) + (3 \times 1)$

Parameters: $(5 \times 3) + 3 = 18$

Output Layer (3 inputs → 1 neuron):

$Z^{[3]} = W^{[3]} \cdot A^{[2]} + b^{[3]}$

$(1 \times 100) = (1 \times 3) \cdot (3 \times 100) + (1 \times 1)$

Parameters: $(3 \times 1) + 1 = 4$

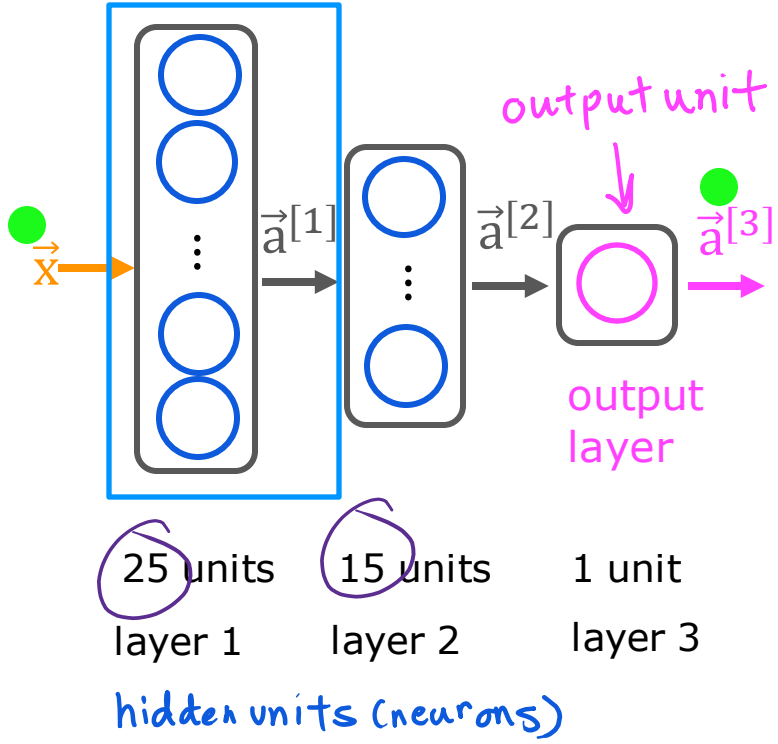
Total Trainable Parameters:

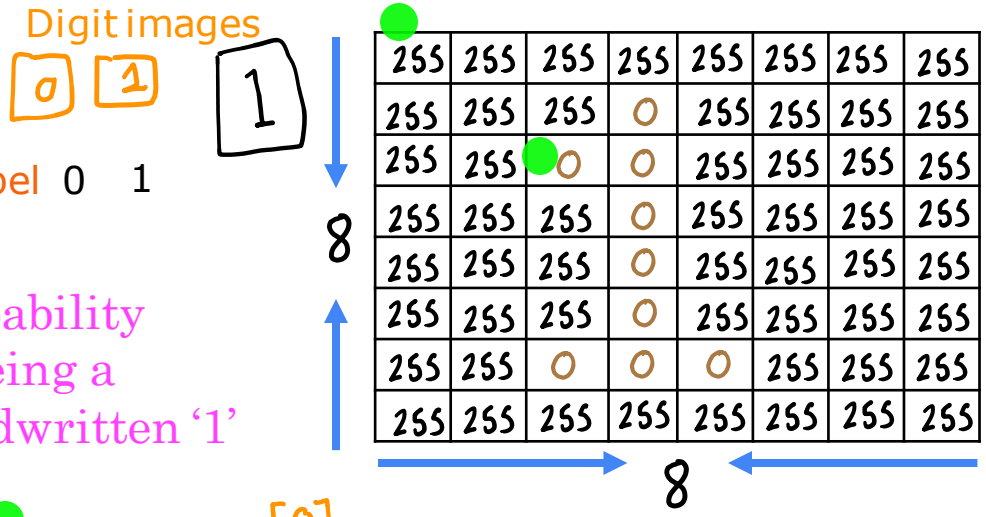
$325 + 18 + 4 = \mathbf{347}$

Neural Network Model

Inference: making predictions (forward propagation)

Handwritten digit recognition

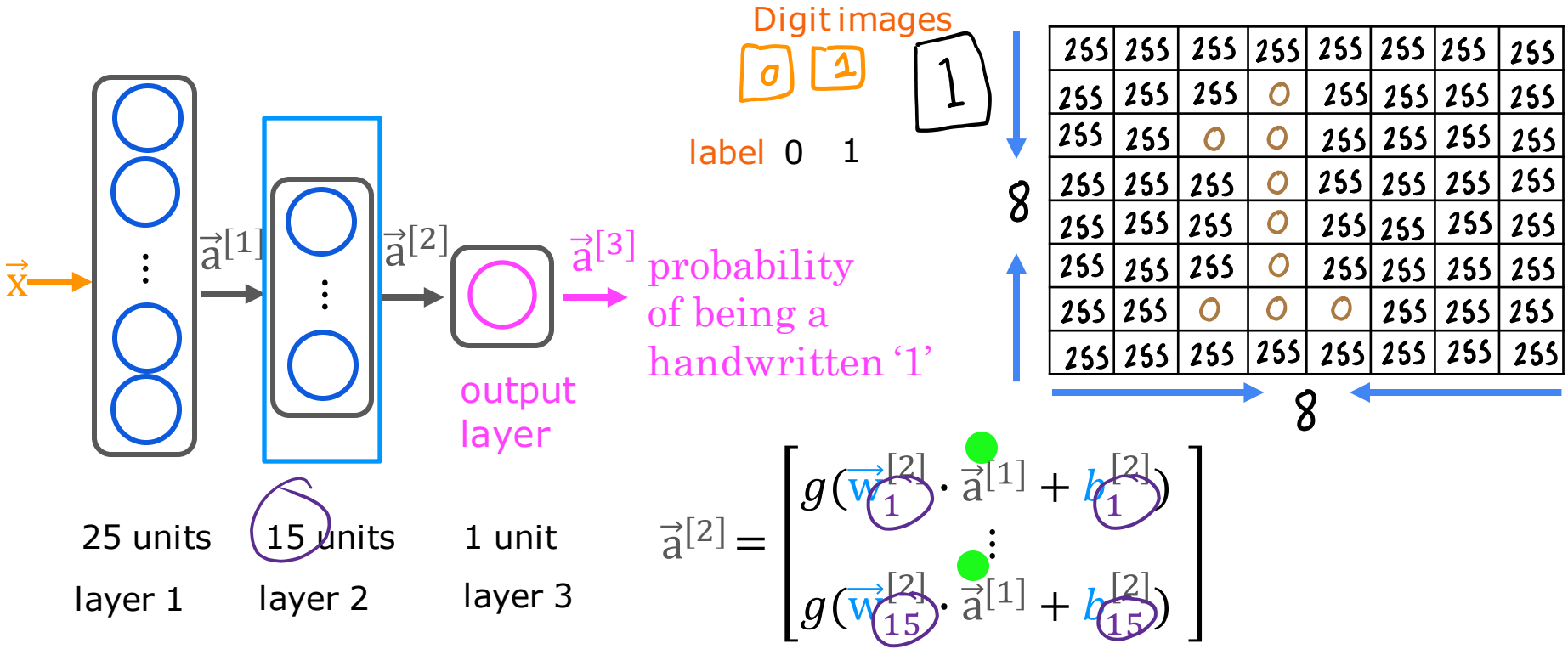




$$a^{[0]}$$

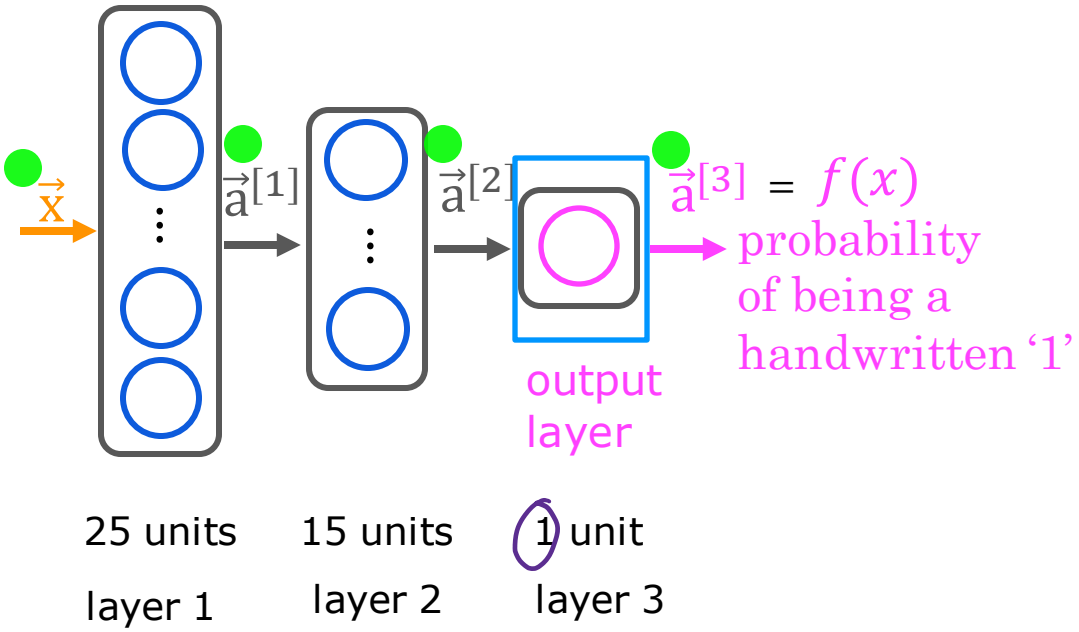
$$\vec{a}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix}$$

Handwritten digit recognition



Handwritten digit recognition

forward propagation



$$\vec{a}^{[3]} = \left[g\left(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]} \right) \right]$$

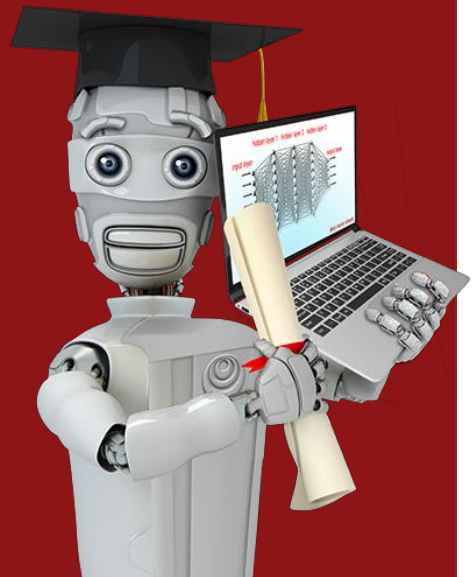
$is\ a_1^{[3]} \geq 0.5?$

yes → $\hat{y} = 1$ image is digit 1

no → $\hat{y} = 0$ image isn't digit 1

DeepLearning.AI

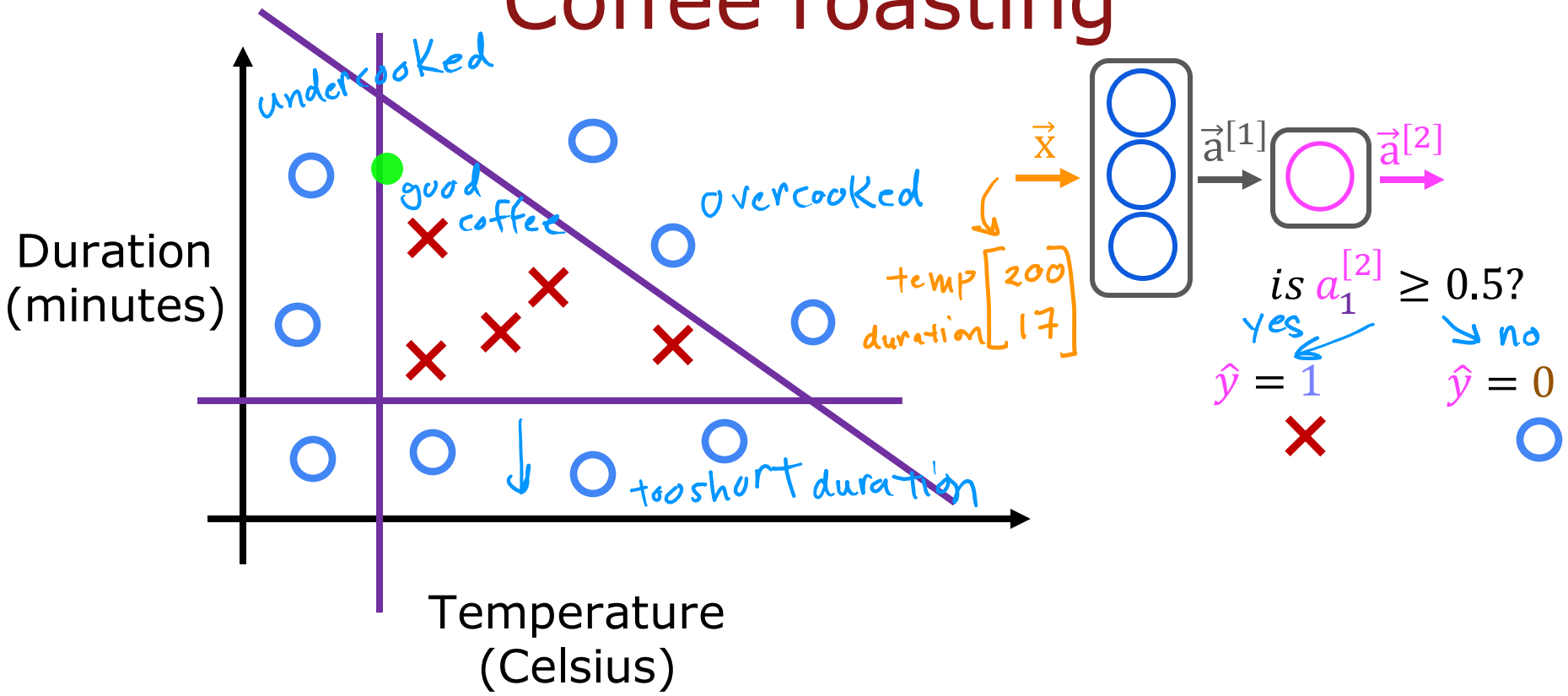
Stanford
ONLINE

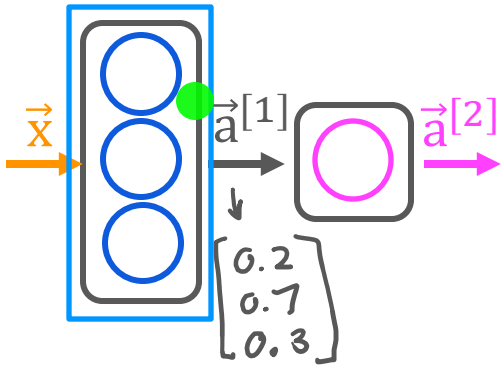


TensorFlow implementation

Inference in Code

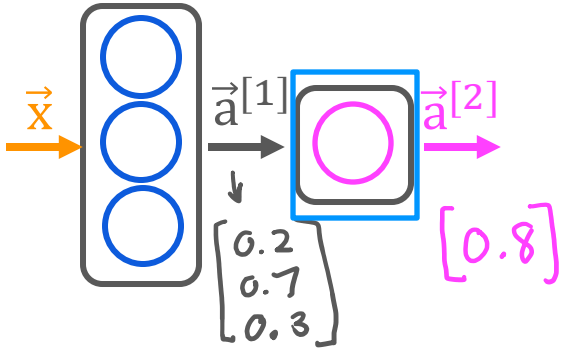
Coffee roasting






```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

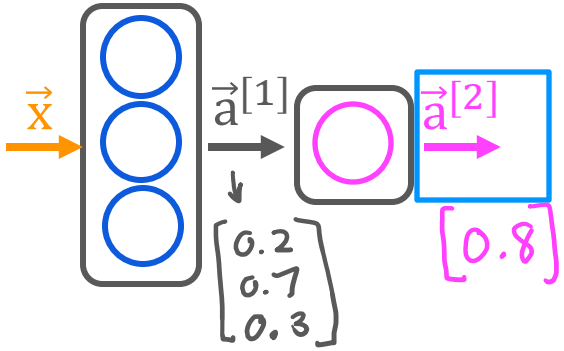
Build the model using TensorFlow

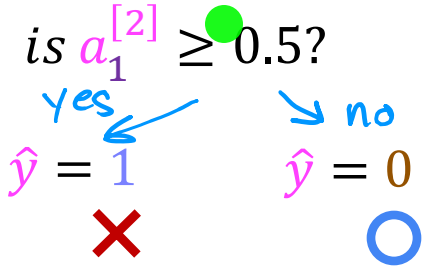



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

Build the model using TensorFlow



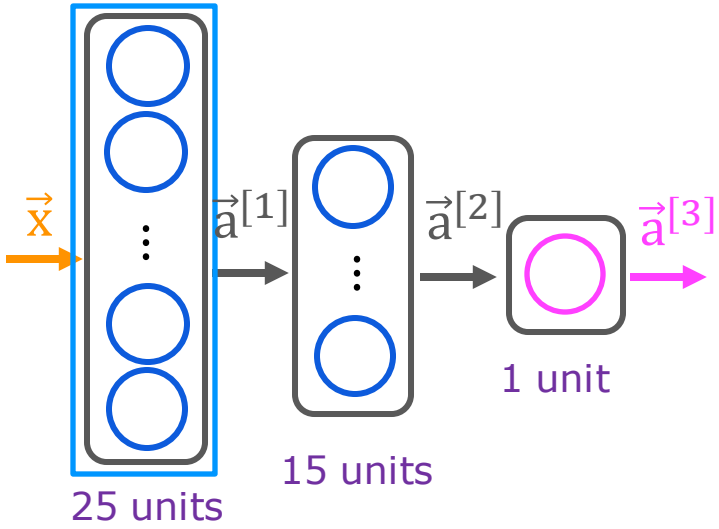



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

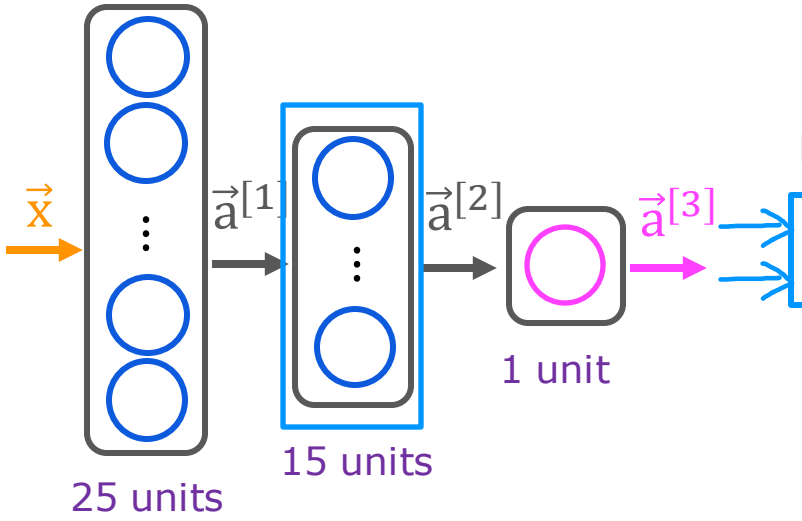
```
if a2 >= 0.5:
  yhat = 1
else:
  yhat = 0
```

Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)
```

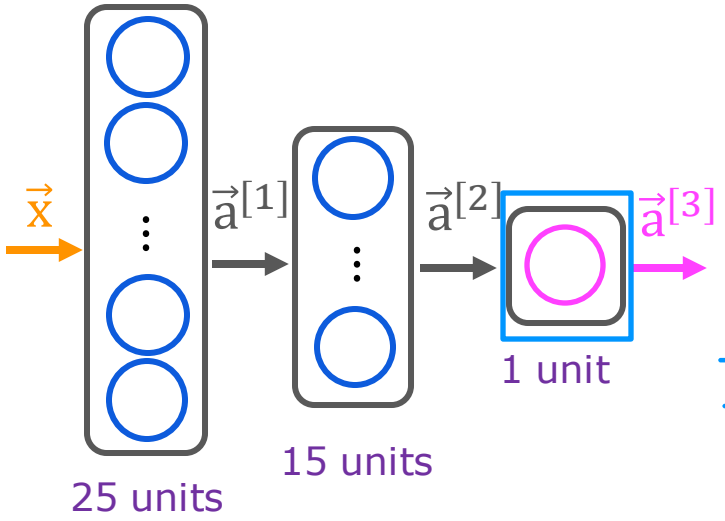
Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)
```

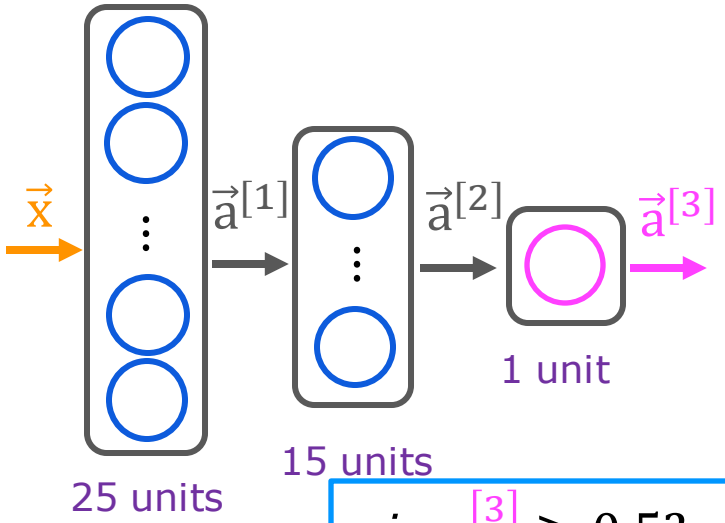
```
layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)
```

Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)

layer_3 = Dense(units=1, activation='sigmoid')
a3 = layer_3(a2)
```

Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)

layer_3 = Dense(units=1, activation='sigmoid')
a3 = layer_3(a2)
```

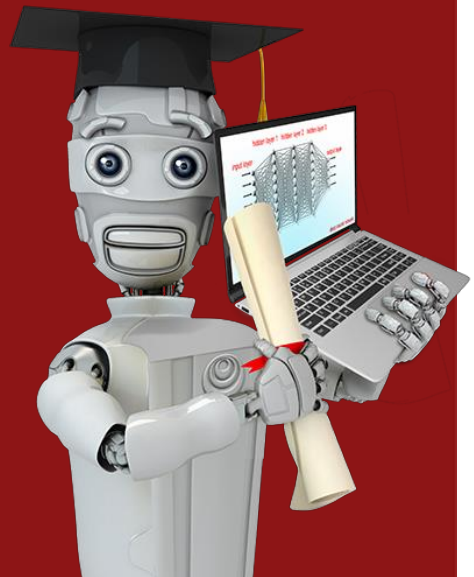
$is\ a_1^{[3]} \geq 0.5?$

$\hat{y} = 1$ ✗ $\hat{y} = 0$ ○

```
if a3 >= 0.5:
  yhat = 1
else:
  yhat = 0
```

DeepLearning.AI

Stanford
ONLINE



TensorFlow implementation

Data in TensorFlow

Feature vectors

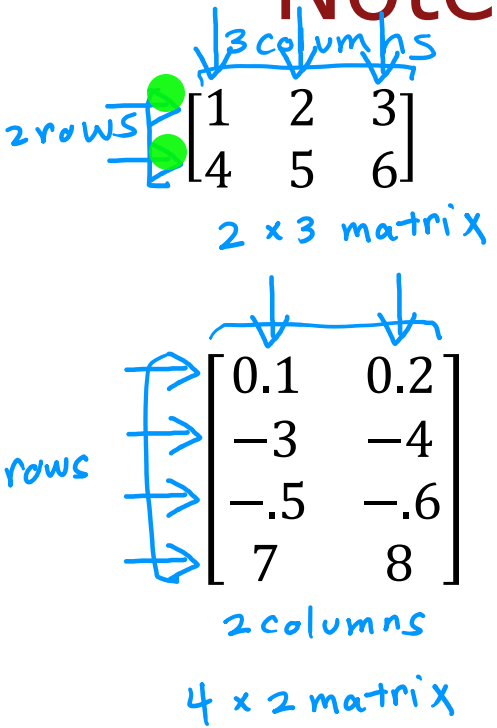
temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
…	…	…

```
x = np.array([[200.0, 17.0]])
```

```
[[200.0, 17.0]]
```

Why?

Note about numpy arrays




```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
[[1, 2, 3],
 [4, 5, 6]]
```

```
x = np.array([[0.1, 0.2],
              [-3.0, -4.0,],
              [-0.5, -0.6,],
              [7.0, 8.0,]])
```

```
[[0.1, 0.2],
 [-3.0, -4.0,],
 [-0.5, -0.6,],
 [7.0, 8.0,]]
```

2D array

2 x 3

4 x 2

1 x 2

2 x 1

Note about numpy arrays

```
x = np.array([[200, 17]])
```

$\begin{bmatrix} 200 & 17 \end{bmatrix}$

1 x 2

```
x = np.array([[200],
              [17]])
```

$\begin{bmatrix} 200 \\ 17 \end{bmatrix}$

2 x 1

```
x = np.array([200,17])
```

1D
"Vector"

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
…	…	…

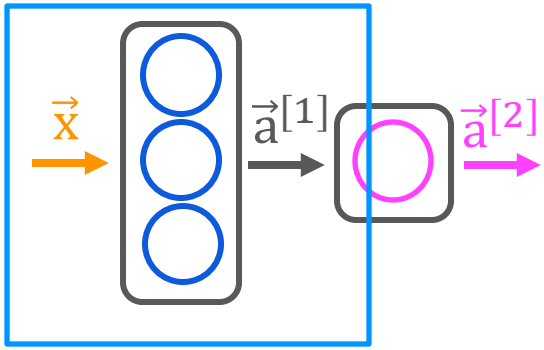
```
x = np.array([[200.0, 17.0]])
```

```
[[200.0, 17.0]]
```

1 x 2

$$\begin{bmatrix} 200.0 & 17.0 \end{bmatrix}$$

Activation vector




```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

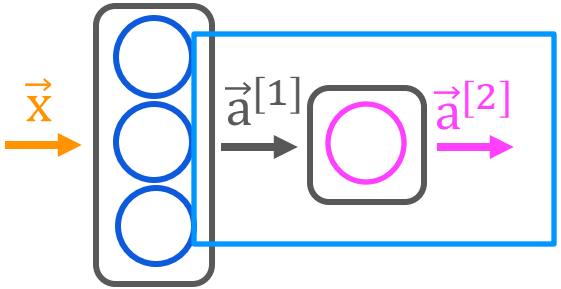
[0.2, 0.7, 0.3] 1 x 3 matrix

```
tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)
```

```
a1.numpy()
```

```
array([[1.4661001, 1.125196 , 3.2159438]], dtype=float32)
```

Activation vector




```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

[[0.8]]

1 x 1

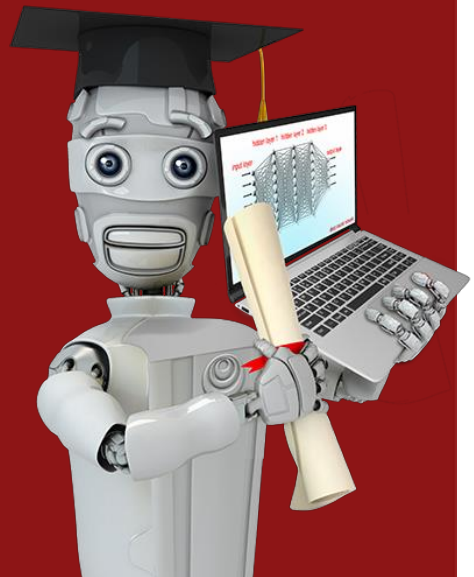
```
tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
```

```
a2.numpy()
```

```
array([[0.8]], dtype=float32)
```

DeepLearning.AI

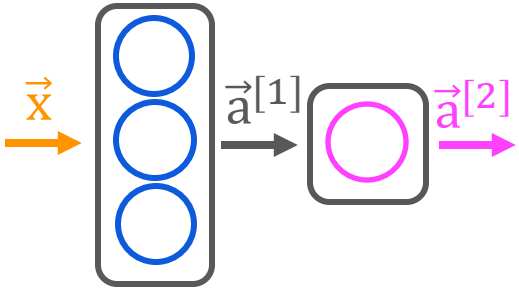
Stanford
ONLINE



TensorFlow implementation

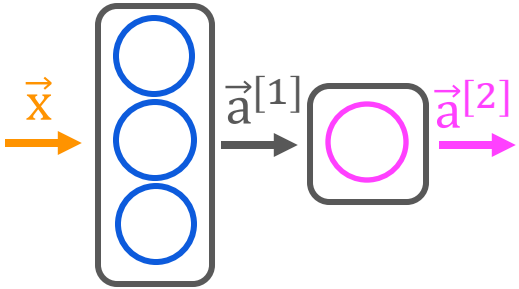
Building a neural network

What you saw earlier




```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation="sigmoid")
a1 = layer_1(x)

layer_2 = Dense(units=1, activation="sigmoid")
a2 = layer_2(a1)
```

Building a neural network architecture



		y
200	17	1
120	5	0
425	20	0
212	18	1

layer1

```
layer_1 = Dense(units=3, activation="sigmoid")
layer_2 = Dense(units=1, activation="sigmoid")
model = Sequential([layer_1, layer_2])
```

layer2

```
x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0]])
```

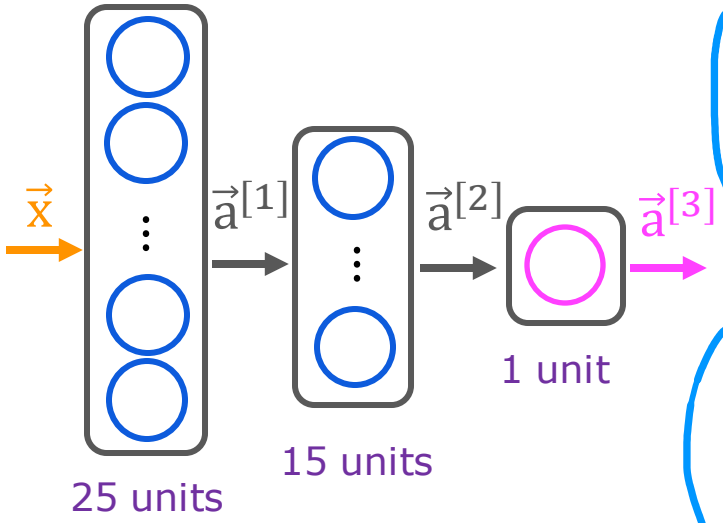
4 x 2

targets

```
y = np.array([1,0,0,1])

model.compile(...)
model.fit(x,y)

model.predict(x_new)
```

← more about this next week!

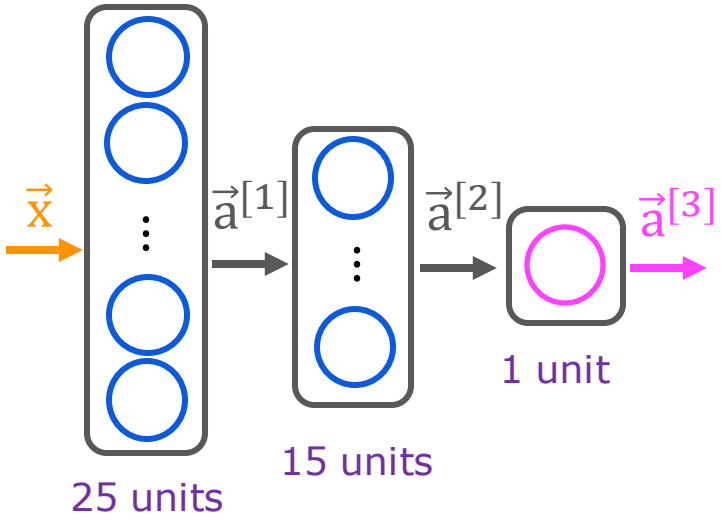
Digit classification model




```
layer_1 = Dense(units=25, activation="sigmoid")
layer_2 = Dense(units=15, activation="sigmoid")
layer_3 = Dense(units=1, activation="sigmoid")
model = Sequential([layer_1, layer_2, layer_3])
model.compile(...)
x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184]])
y = np.array([1,0])
model.fit(x,y)
model.predict(x_new)
```

more about this next week!

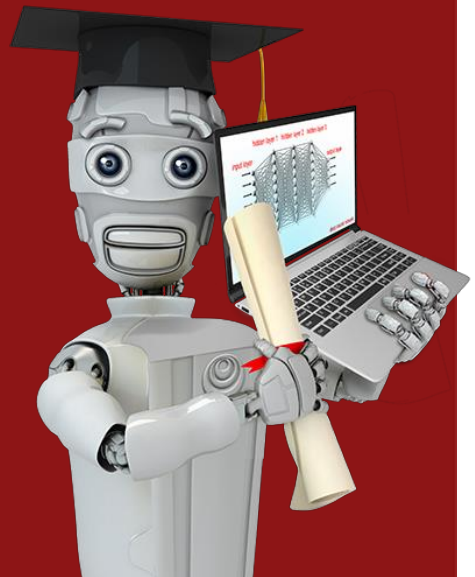
Digit classification model




```
model = Sequential([
    Dense(units=25, activation="sigmoid"),
    Dense(units=15, activation="sigmoid"),
    Dense(units=1, activation="sigmoid")])

model.compile(...)

x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184])
y = np.array([1,0])

model.fit(x,y)

model.predict(x_new)
```

DeepLearning.AI

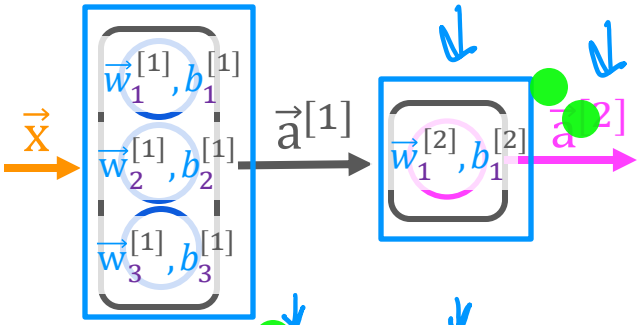
Stanford
ONLINE



Neural network implementation in Python

Forward prop in a single layer

forward prop (coffee roasting model)



$a_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$

```
w2_1 = np.array([-7, 8])
b2_1 = np.array([3])
z2_1 = np.dot(w2_1,a1)+b2_1
a2_1 = sigmoid(z1_1)
```

$w_1^{[2]}$ w2_1

```
x = np.array([200, 17])
```

1D arrays

$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$ $a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$ $a_3^{[1]} = g(\vec{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$

```
w1_1 = np.array([1, 2])
b1_1 = np.array([-1])
z1_1 = np.dot(w1_1,x)+ b
a1_1 = sigmoid(z1_1)
```

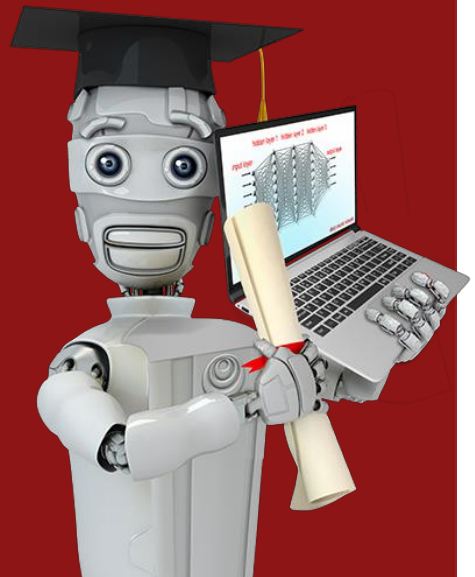
```
w1_2 = np.array([-3, 4])
b1_2 = np.array([1])
z1_2 = np.dot(w1_2,x)+b
a1_2 = sigmoid(z1_2)
```

```
w1_3 = np.array([5, -6])
b1_3 = np.array([2])
z1_3 = np.dot(w1_3,x)+b
a1_3 = sigmoid(z1_3)
```

```
a1 = np.array([a1_1, a1_2, a1_3])
```

DeepLearning.AI

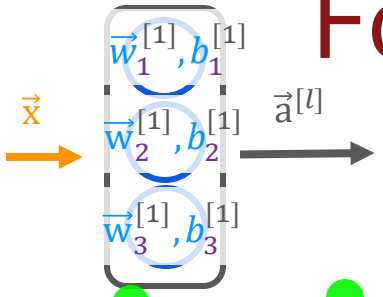
Stanford
ONLINE



Neural network implemenation in Python

General implementation of forward propagation

Forward prop in NumPy



$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ $\vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix}$ $\vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$

```
W = np.array([
     [1,  -3,  5]
     [2,   4, -6]])
```
2 by 3

$b_1^{[l]} = -1$ $b_2^{[l]} = 1$ $b_3^{[l]} = 2$

```
b = np.array([-1, 1, 2])
```

$\vec{a}^{[0]} = \vec{x}$

```
a_in = np.array([-2, 4])
```

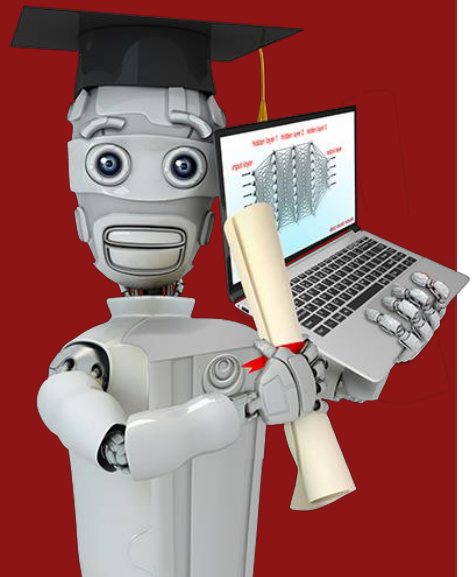
```
def dense(a_in,W,b, g):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:,j]
        z = np.dot(w,a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```
3 [0,0,0] 0,1,2

```
def sequential(x):
    a1 = dense(x,W1,b1,g)
    a2 = dense(a1,W2,b2,g)
    a3 = dense(a2,W3,b3,g)
    a4 = dense(a3,W4,b4,g)
    f_x = a4
    return f_x
```
$a^{[1]}$ $a^{[2]}$

capital W refers to a matrix

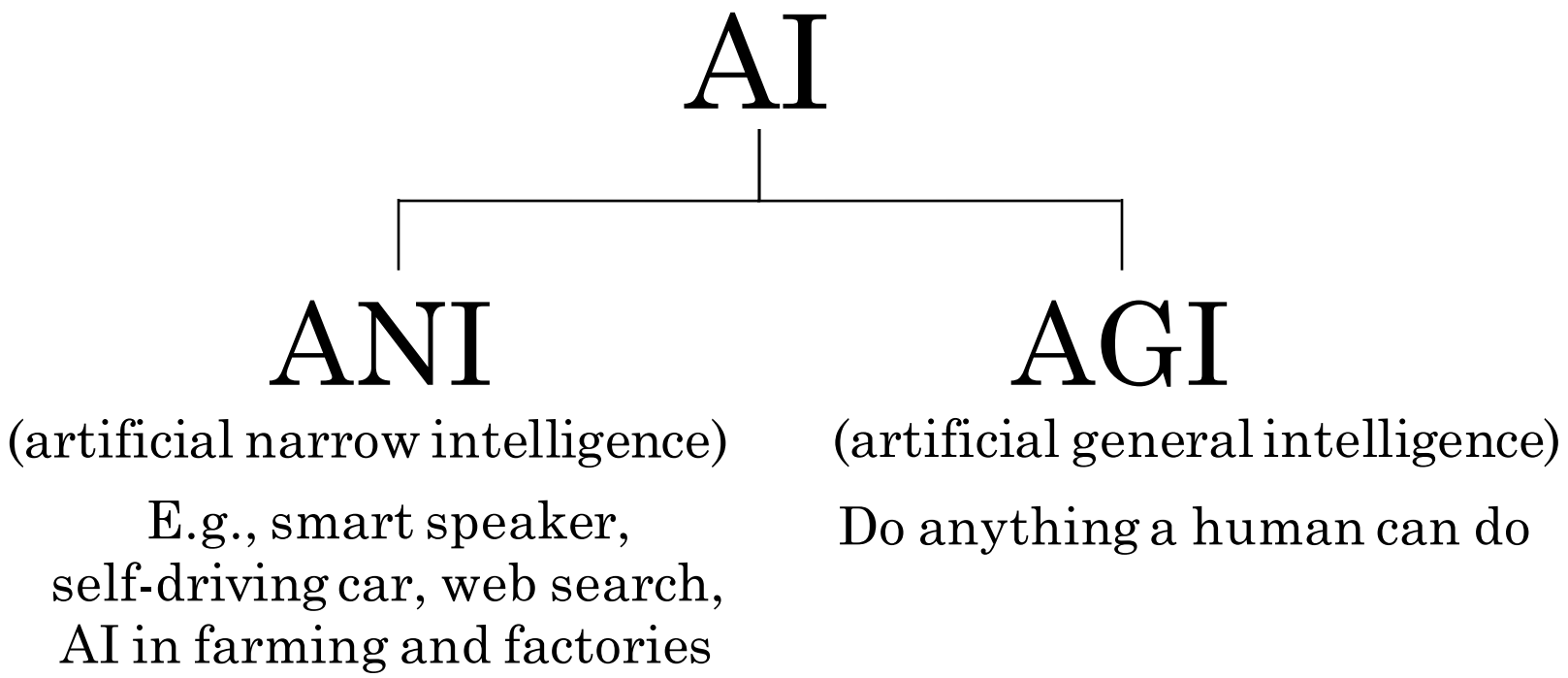
DeepLearning.AI

Stanford
ONLINE



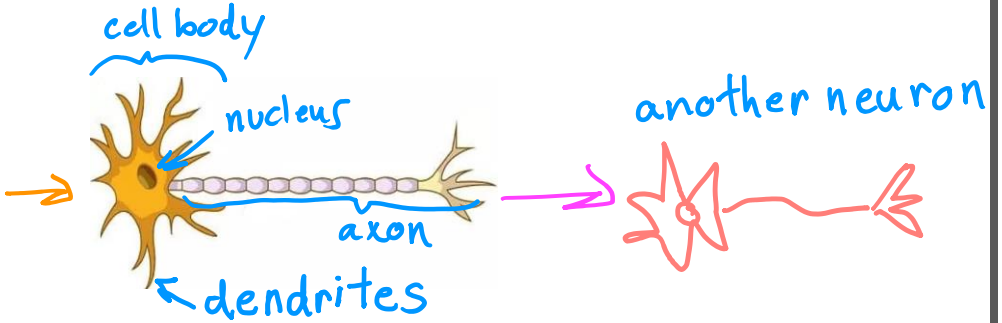
Speculations on artificial general intelligence (AGI)

Is there a path to AGI?


AI
ANI
(artificial narrow intelligence)
E.g., smart speaker,
self-driving car, web search,
AI in farming and factories
AGI
(artificial general intelligence)
Do anything a human can do

Biological neuron

inputs outputs



Simplified mathematical model of a neuron

inputs outputs

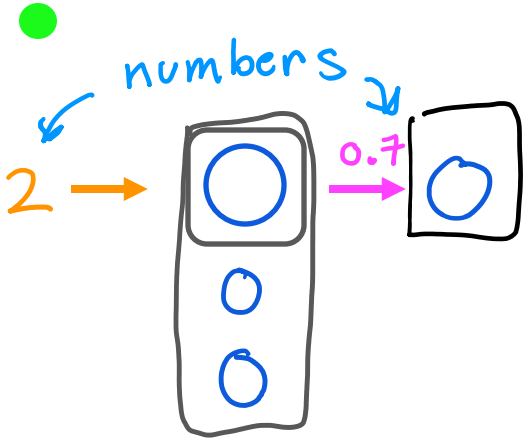
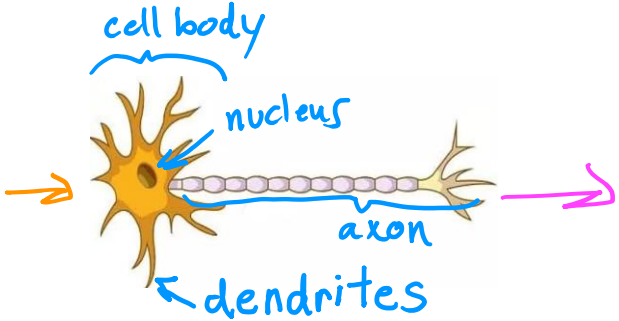


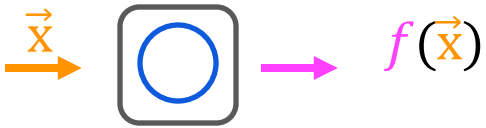
image source: https://biologydictionary.net/sensory-neuron/

Neural network and the brain

Can we mimic the human brain?

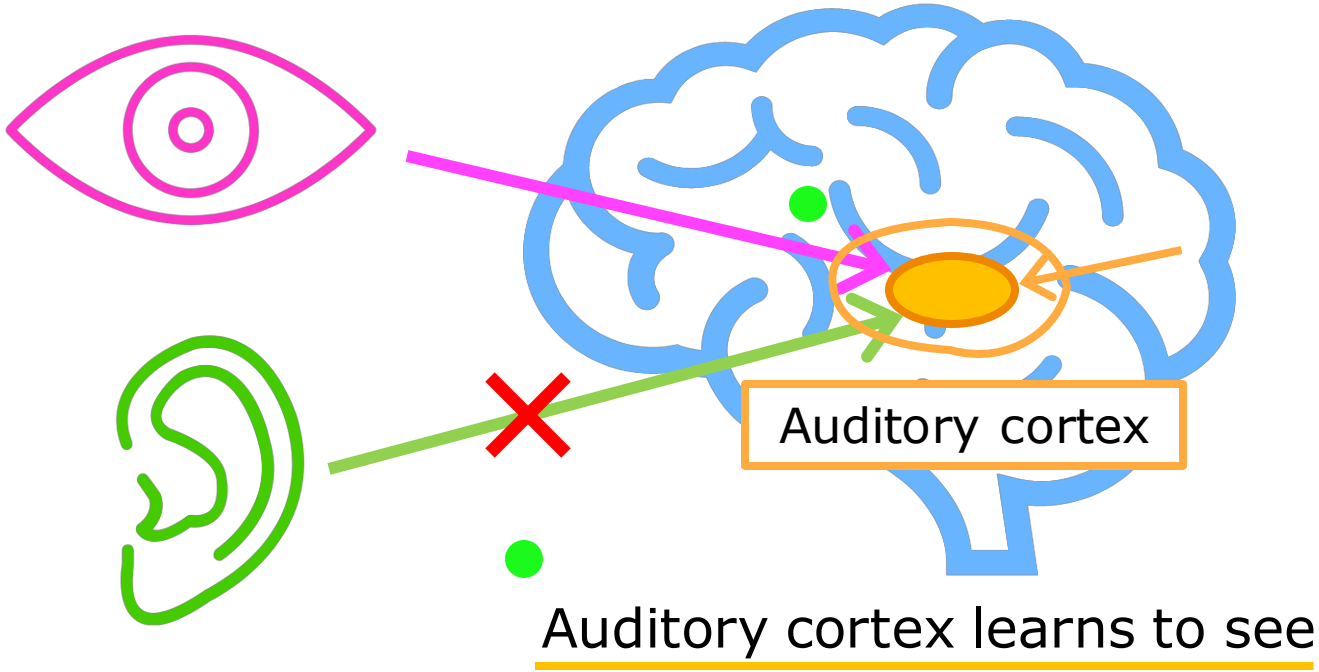


vs



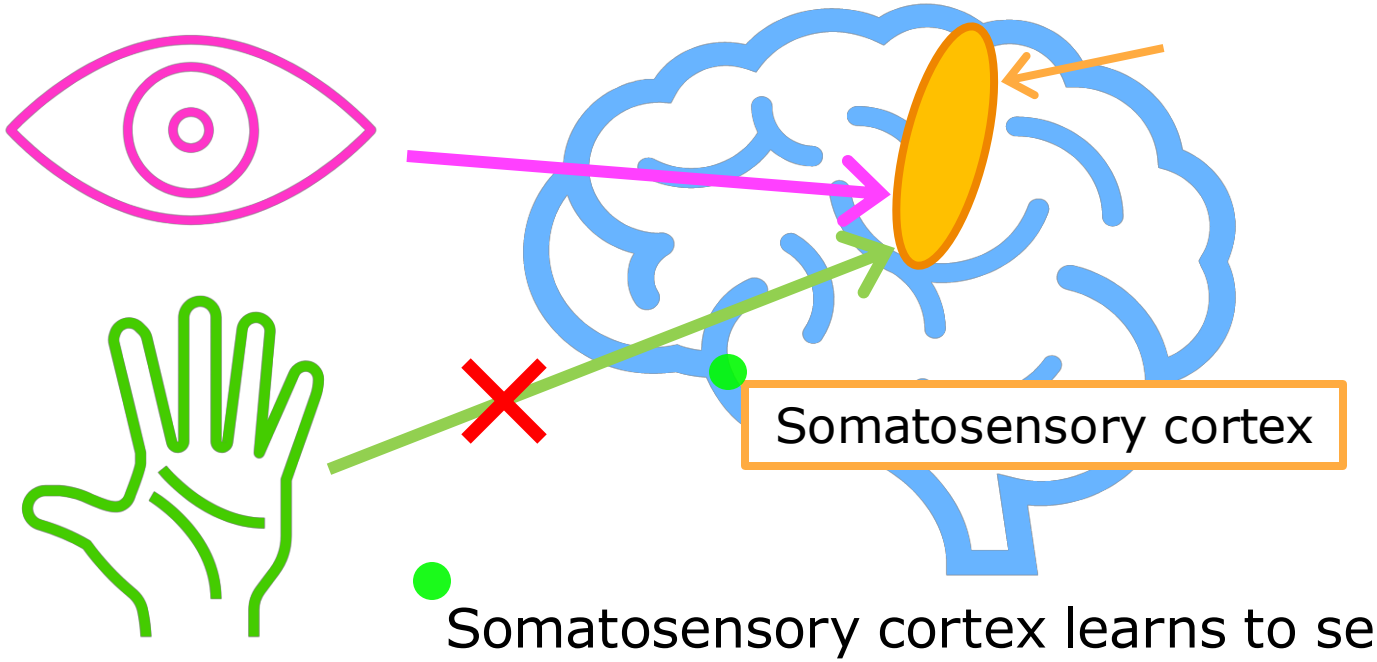
We have (almost) no idea how the brain works

The ”one learning algorithm” hypothesis



[Roe et al., 1992]

The ”one learning algorithm” hypothesis



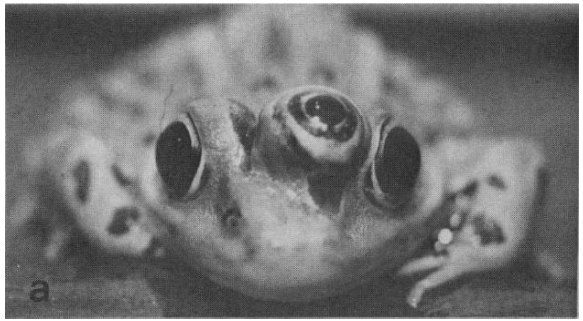
[Metin & Frost, 1989]

Sensor representations in the brain

Seeing with your tongue

Human echolocation (sonar)

Haptic belt: Direction sense

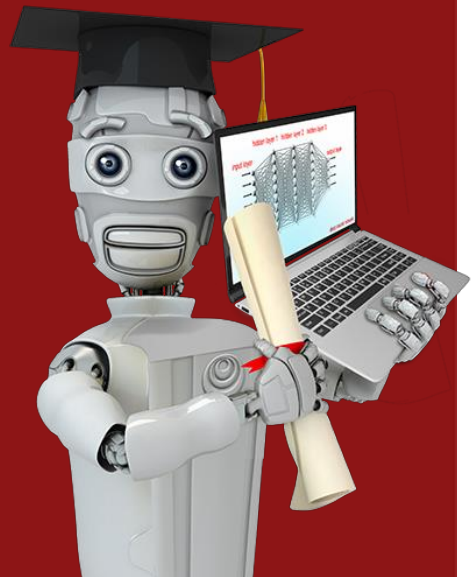


Implanting a 3rd eye

[BrainPort; Welsh & Blasch, 1997; Nagel et al., 2005; Constantine-Paton & Law, 2009]

DeepLearning.AI

Stanford
ONLINE



Vectorization (optional)

How neural networks are implemented efficiently

For loops vs. vectorization

```
x = np.array([200, 17])

W = np.array([[1, -3, 5],
              [-2, 4, -6]])

b = np.array([-1, 1, 2])

def dense(a_in,W,b):
    a_out = np.zeros(units)
    for j in range(units):
      w = W[:,j]
      z = np.dot(w,x) + b[j]
      a[j] = g(z)
  return a
```

[1,0,1]

Vectorized

```
X = np.array([[200, 17]])

W = np.array([[1, -3, 5],
              [-2, 4, -6]])

B = np.array([[-1, 1, 2]])

def dense(A_in,W,B):
    Z = np.matmul(A_in,W) + B
    A_out = g(Z)
    return A_out
```

[[1,0,1]]

2D array

same

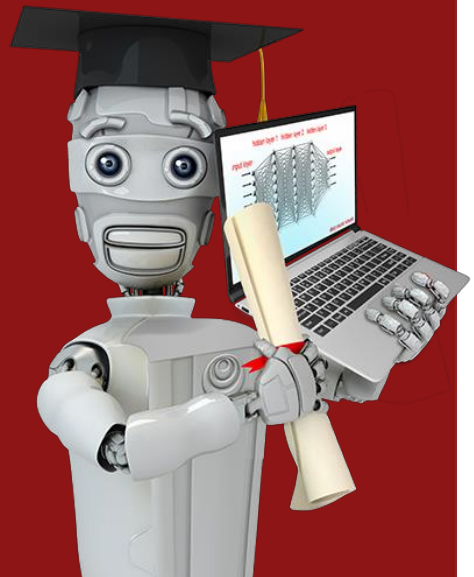
1x3 2D array

all 2D arrays

matrix multiplication

DeepLearning.AI

Stanford
ONLINE



Vectorization (optional)

Matrix multiplication

Dot products

example

$\begin{bmatrix}1\\2\end{bmatrix} \cdot \begin{bmatrix}3\\4\end{bmatrix}$

$z = (1 \times 3) + (2 \times 4)$

$3 + 8$

11

in general

$\begin{bmatrix}\uparrow\\ \vec{a} \\ \downarrow\end{bmatrix} \cdot \begin{bmatrix}\uparrow\\ \vec{w} \\ \downarrow\end{bmatrix}$

$z = \vec{a} \cdot \vec{w}$

transpose

$\vec{a} = \begin{bmatrix}1\\2\end{bmatrix}$

$\vec{a}^T = [1 \quad 2]$

vector vector multiplication

$[\leftarrow \ \vec{a}^T \ \rightarrow] \begin{bmatrix}\uparrow\\ \vec{w} \\ \downarrow\end{bmatrix}$

$1 \times 2 \qquad 2 \times 1$

$z = \vec{a}^T \vec{w}$

equivalent

useful for understanding matrix multiplication

Vector matrix multiplication

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \vec{a}^T = \begin{bmatrix} 1 & 2 \end{bmatrix} \qquad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$$

$$Z = \vec{a}^T W \quad \begin{bmatrix} \leftarrow & \vec{a}^T & \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow & \uparrow \\ \vec{w}_1 & \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

1 by 2

$$Z = \begin{bmatrix} \vec{a}^T \vec{w}_1 & a^T \vec{w}_2 \end{bmatrix}$$

$$(1 * 3) + (2 * 4) \qquad (1 * 5) + (2 * 6)$$

$3 + 8$ $\quad$ $5 + 12$

11 $\quad$ 17

$$Z = [11 \quad 17]$$

matrix matrix multiplication

$$A = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} \leftarrow & \vec{a}_1^T & \rightarrow \\ \leftarrow & \vec{a}_2^T & \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow & \uparrow \\ \vec{w}_1 & \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

rows

columns

$$= \begin{bmatrix} \vec{a}_1^T \vec{w}_1 & \vec{a}_1^T \vec{w}_2 \\ \vec{a}_2^T \vec{w}_1 & \vec{a}_2^T \vec{w}_2 \end{bmatrix}$$

row 1 col 1

row 2 col 1

row 1 col 2

row 2 col 2

$(-1 \times 3) + (-2 \times 4)$

$-3 + -8$

-11

$(-1 \times 5) + (-2 \times 6)$

$-5 + -12$

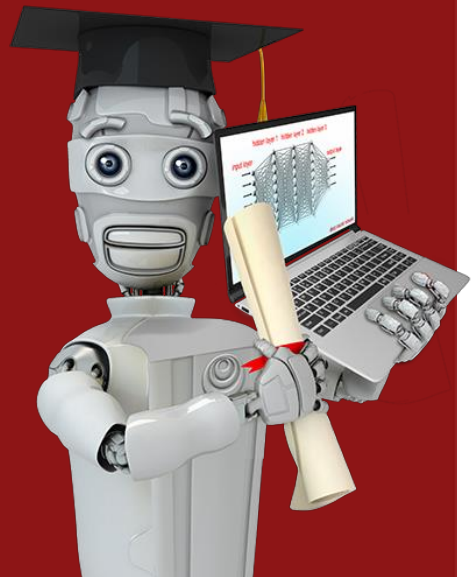
-17

$$= \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$$

general rules for matrix multiplication
↳ next video!

DeepLearning.AI

Stanford
ONLINE



Vectorization (optional)

Matrix multiplication rules

Matrix multiplication rules

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix}$$

$\vec{a}_1^T$ $\vec{a}_2^T$ $\vec{a}_3^T$

$\vec{w}_1$ $\vec{w}_2$ $\vec{w}_3$ $\vec{w}_4$

$$Z = \mathbf{A}^T\mathbf{W} = \begin{bmatrix} & & & \\ & & & \\ & & & \end{bmatrix}$$

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} & & & \\ & & & \\ & & & \end{bmatrix}$$

3 by 4 matrix

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

0.5 + 1.2

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 + 8) = -23$$

-7 + -16

Matrix multiplication rules

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3 by 4 matrix

$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$

row 3 column 2

$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$

$0.5 + 1.2$

row 2 column 3?

$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 + 8) = -23$

$-7 + -16$

Matrix multiplication rules

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \mathbf{Z} = \mathbf{A}^T\mathbf{W} = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3 x 2 2 x 4

can only take dot products of vectors that are same length

[0.1 0.2] length 2

$\begin{bmatrix} 5 \\ 6 \end{bmatrix}$ length 2

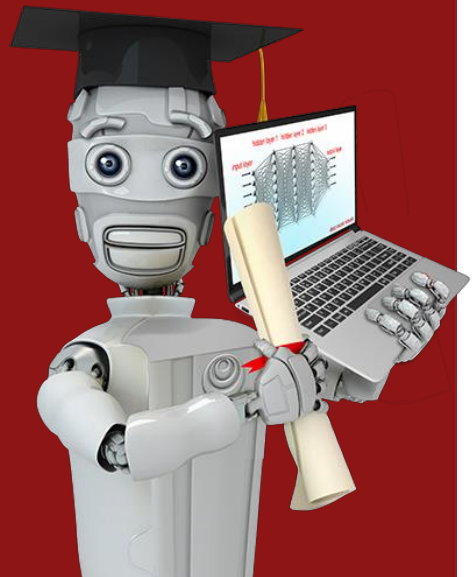
3 by 4 matrix

↳ same # rows as A^T

same # columns as W

DeepLearning.AI

Stanford
ONLINE



Vectorization (optional)

Matrix multiplication code

Matrix multiplication in NumPy

$$A = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad W = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad Z = A^T W = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

```
A=np.array([1,-1,0.1],
           [2,-2,0.2]])
```

```
W=np.array([3,5,7,9],
           [4,6,8,0])
```

```
Z = np.matmul(AT,W)
```

or

```
Z = AT @ W
```

```
AT=np.array([1,2],
            [-1,-2],
            [0.1,0.2])
```

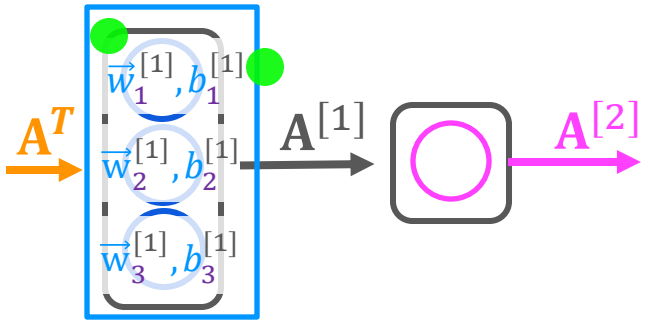
result

```
[[11,17,23,9],
 [-11,-17,-23,-9],
 [1.1,1.7,2.3,0.9]
]
```

```
AT=A.T
```

transpose

Dense layer vectorized



$$\mathbf{A}^T = [200 \quad 17] \qquad 1 \times 2$$

$$\mathbf{W} = \begin{bmatrix} 1 & -3 & 5 \\ -2 & 4 & -6 \end{bmatrix} \qquad 2 \times 3$$

$$\vec{\mathbf{b}} = [-1 \quad 1 \quad 2] \qquad 1 \times 3$$

$$\mathbf{Z} = \mathbf{A}^T\mathbf{W} + \mathbf{B}$$

$$[165 \quad -531 \quad 900]$$

$$z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]}$$

$$\mathbf{A} = g(\mathbf{Z})$$

$$[1 \quad 0 \quad 1]$$

A

```
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([[-1, 1, 2]])
```

a_in

```
def dense(AT,W,b,g):
  z = np.matmul(AT,W) + b
  a_out = g(z)
  return a_out
```

a_in

```
[[1,0,1]]
```